MAVEN WORKSHOP · COHORT 2

Introduction to AI agent memory.

Cut what your agents cost per turn while improving how well they perform.

AGENDA

Four hours, six modules, one memory-aware agent

Opening	Why agents need memory, the agent loop, the cost of forgetting
01 What is agent memory	Definition, the taxonomy, five lenses, eight memory types
02 The agent memory landscape	Personal agents, Anthropic, OpenAI, clouds and startups
Break	10 minutes
03 Implementing memory-aware agents	Context and memory engineering, the reference architecture, extraction
04 Tokenomics	Cost, latency and accuracy, priced per request
Break	10 minutes
05 Memory for coding agents	The MemoRizz plugin for Claude Code and Codex, live
06 Continual learning	Two ends of memory engineering, where knowledge belongs

OPENING QUESTION

Why do our agents need memory?

OPENING · WHAT IS AN AGENT

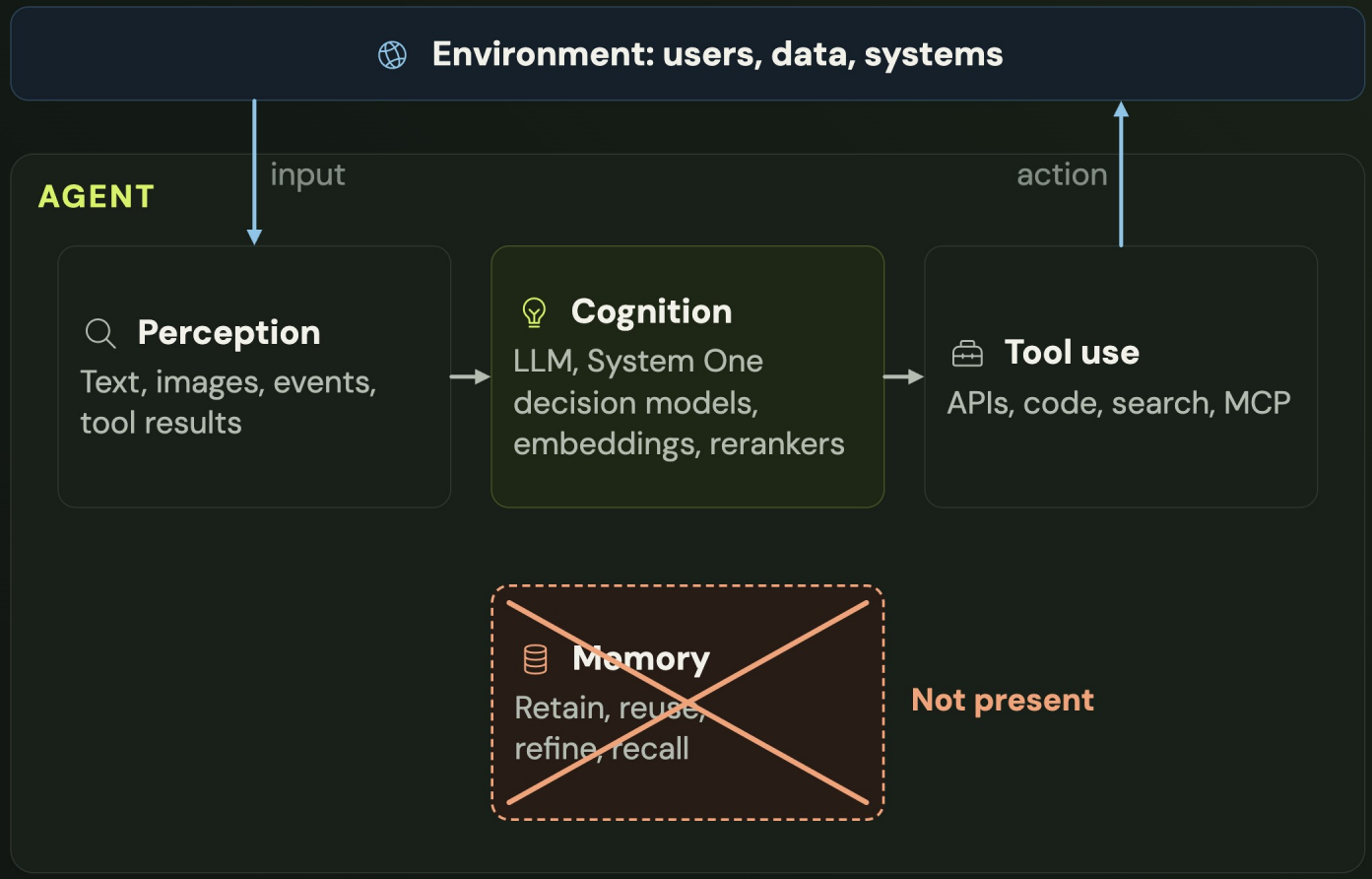
An agent, without memory

An agent is a system that perceives its environment, reasons with models and acts through tools, in a loop, towards a goal.

Cognition is more than the LLM

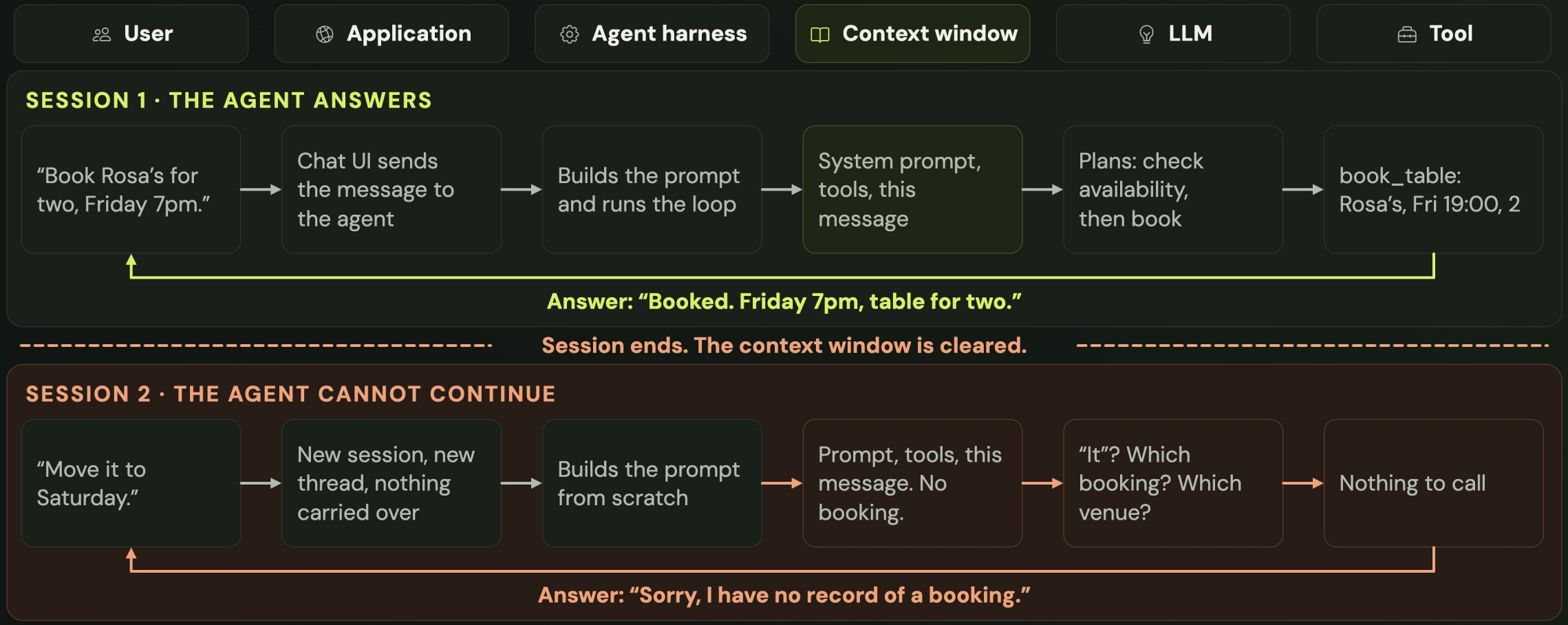
System One decision models, embedding models and rerankers all make judgements inside the loop.

Without memory, every loop starts from zero.



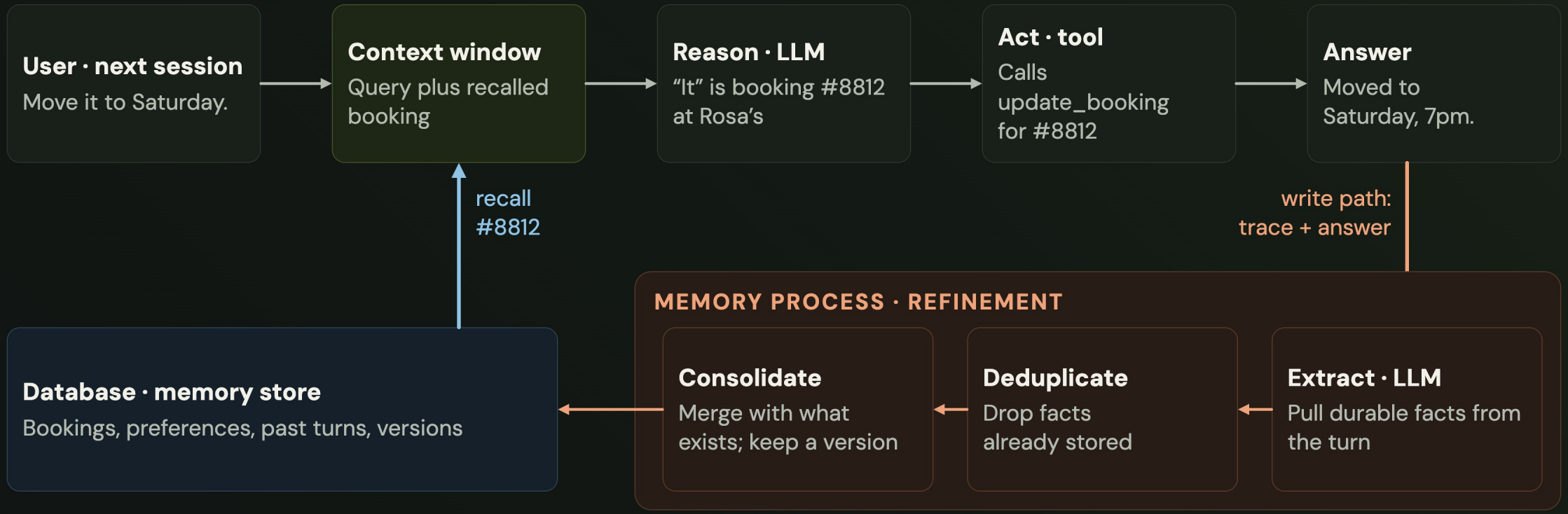
THE AGENT LOOP · WITHOUT MEMORY

The same agent, one session later



THE AGENT LOOP · WITH MEMORY

Memory feeds the window and learns from each answer



Iteration 2 now works: "move it to Saturday" resolves to booking #8812 at Rosa's, 7pm.

OPENING · THE COST OF FORGETTING

Without memory: the same loop, turn by turn

WITHOUT MEMORY

User

Agent harness

replays everything

CONTEXT WINDOW · whole history, every call

turn 1 | turn 2 | turn 3 | turn 4 | turn 5

turn 6 | turn 7 | turn 8 | turn 9 | turn 10

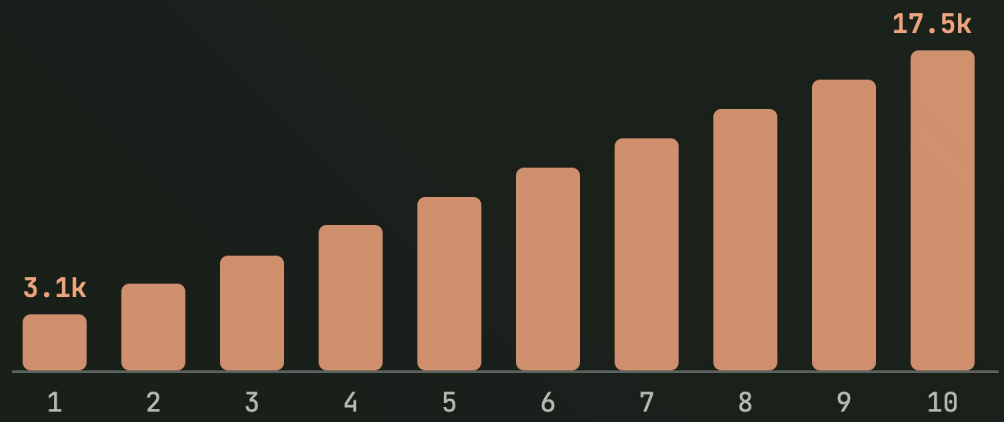
tokens grow every turn

LLM
answers, then forgets

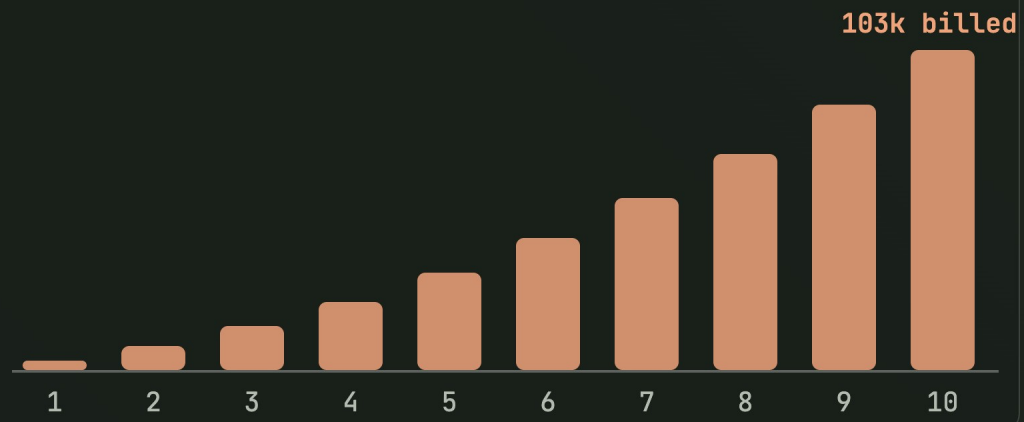
Tokens per turn grow with the conversation. Corrections die with the session.

WHAT IT COSTS, TURN BY TURN

TOKENS SENT PER TURN — 5.6× by turn 10



TOKENS BILLED, CUMULATIVE — the bill grows faster

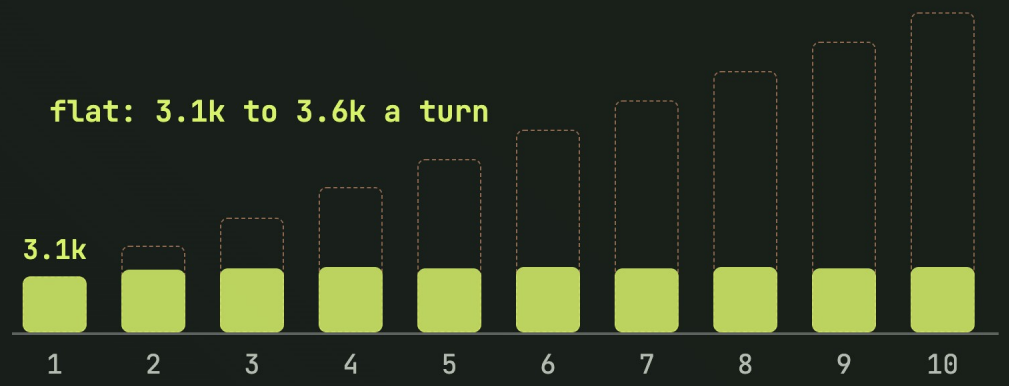


OPENING · THE COST OF FORGETTING

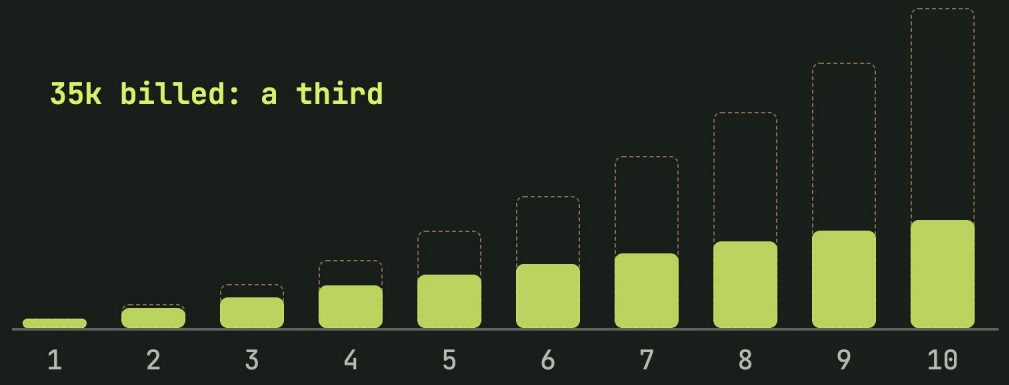
With memory: the same loop, turn by turn

WHAT IT COSTS, TURN BY TURN

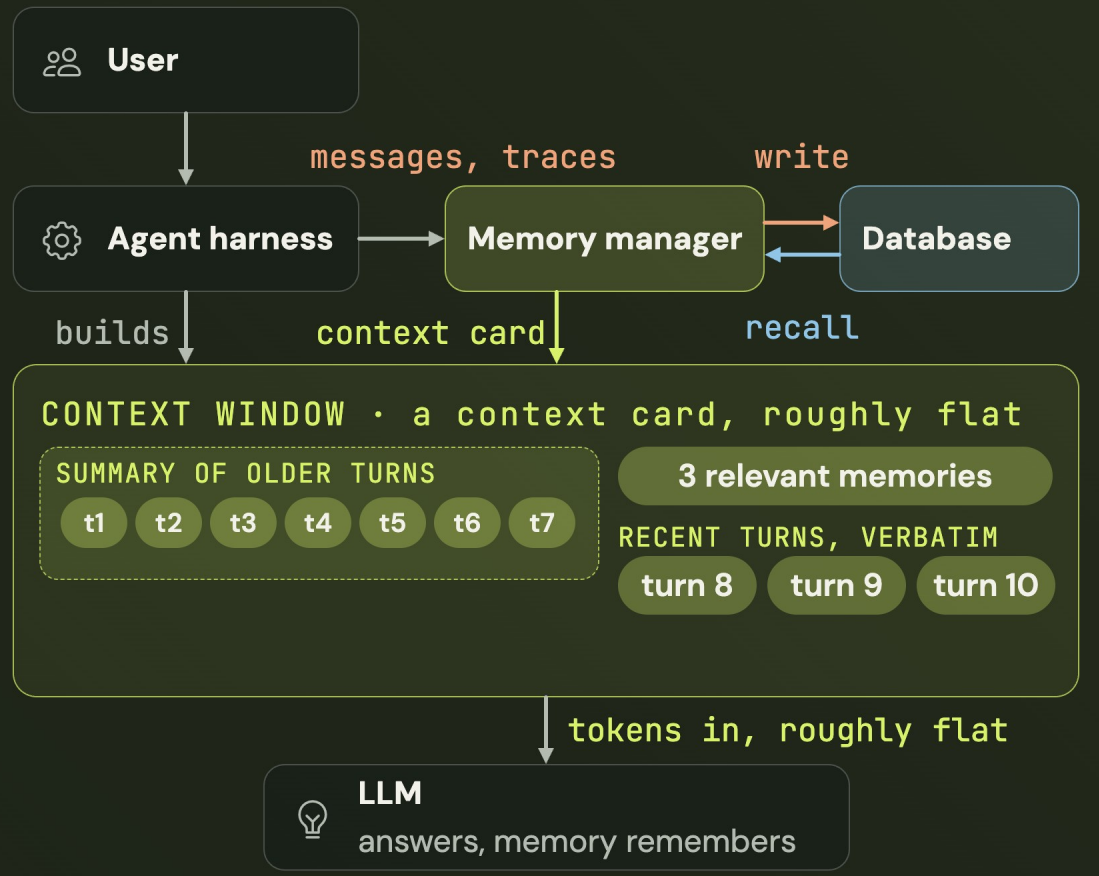
TOKENS SENT PER TURN — dashed: without memory



TOKENS BILLED, CUMULATIVE — dashed: without memory



WITH MEMORY



Tokens per turn stay roughly flat. Corrections become memory and come back next session.

THREE LEVERS, ONE INVOICE

Tokenomics

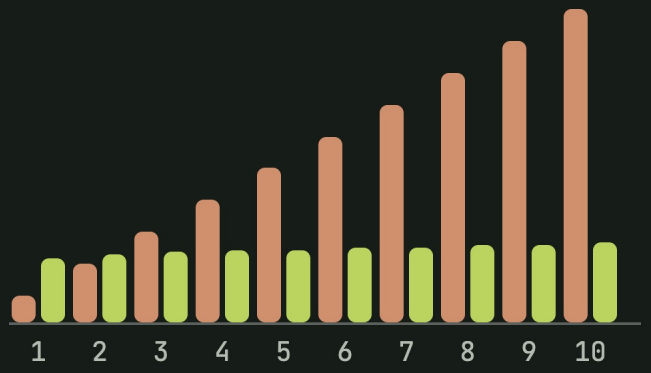
Cost, latency and accuracy, turn by turn. Good agent memory moves all three.

COST

tokens billed per turn

no memory
good memory

cost down



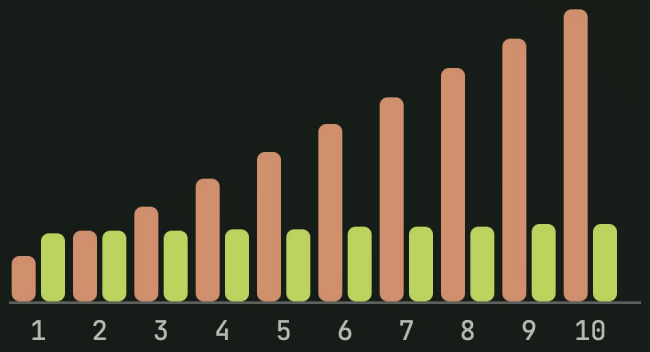
Replaying history means tokens grow every turn. Memory plus a context card keeps the window roughly flat.

LATENCY

time to first token

no memory
good memory

latency down



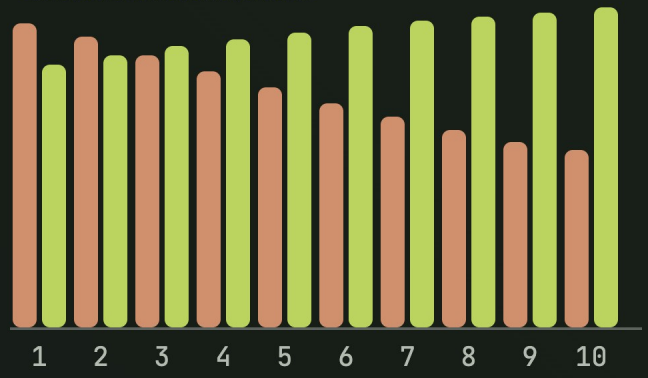
Prefill scales with the window. A flat window keeps the first token, and the p95, where it was on turn one.

ACCURACY

answers that hold up

no memory
good memory

accuracy up



Long context loses facts in the middle and repeats corrected mistakes. Memory brings back the right ones.

The gap between the orange and the lime bars is the bill, the wait, and the mistake. Memory is what closes it.

Illustrative shapes, not measurements. Accuracy drop: LongMemEval, ICLR 2025.

TOKENOMICS, PLUS THE GATE

Why your agent needs memory

Tokenomics is cost, latency and accuracy on one invoice. Governance decides whether you may ship at all.

COST

Tokens grow with every turn

Agents that resend history pay for it on every call. Inference spend rises with token use, and a fleet multiplies it.

LATENCY

Longer prompts, slower answers

Prefill scales with the window. More tokens in means a slower first token and a worse p95, every turn.

ACCURACY

Long context is not memory

Agents repeat corrected mistakes. LongMemEval measured about a 30% accuracy drop for long-context assistants over sustained use.

GOVERNANCE

Memory is enterprise data

It must respect who may see what, expire on purpose, stay where the data lives, and be auditable like any other record.

Memory moves all four, which is why hyperscalers, incumbents and startups are all converging on it.

Accuracy figure: Wu et al., LongMemEval, ICLR 2025.

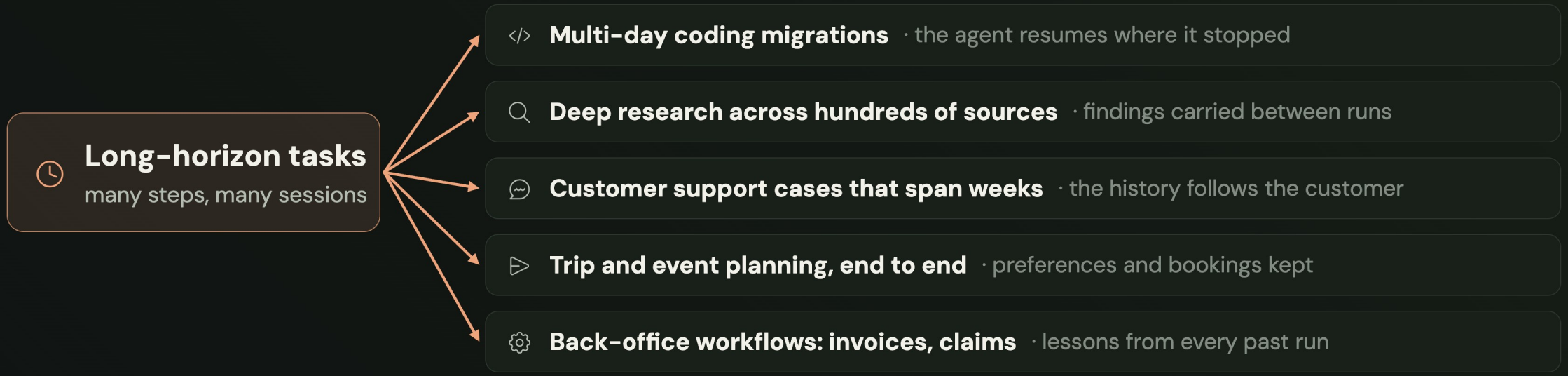
MODULE 01

What is agent memory?

Definition, one taxonomy, five lenses and eight memory types.

DEFINITION

Agent memory is how an agent retains, reuses, refines and recalls what it learns, across turns, sessions and agents, making them applicable for long-horizon tasks.



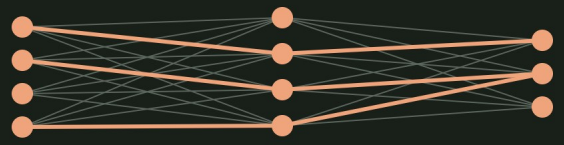
01 · WHAT IS AGENT MEMORY

A working definition

Agent memory is how an agent retains, reuses, refines and recalls what it learns, across turns, sessions and agents, making them applicable for long-horizon tasks.

PARAMETRIC · WEIGHTS

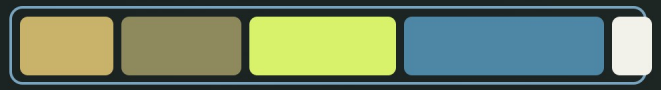
Model weights



READ	every forward pass; nothing to query
WRITE	training and fine-tuning; a batch job
LIVES	on the accelerator, frozen at inference
COST	paid once per model; free per call

WORKING · CONTEXT WINDOW

Context window



prefix · memory · turns · query

READ	attention over every token, every call
WRITE	the harness assembles it each call
LIVES	one call; the KV cache keeps the prefix warm
COST	per input token, every call

PERSISTENT · DATABASE

Database

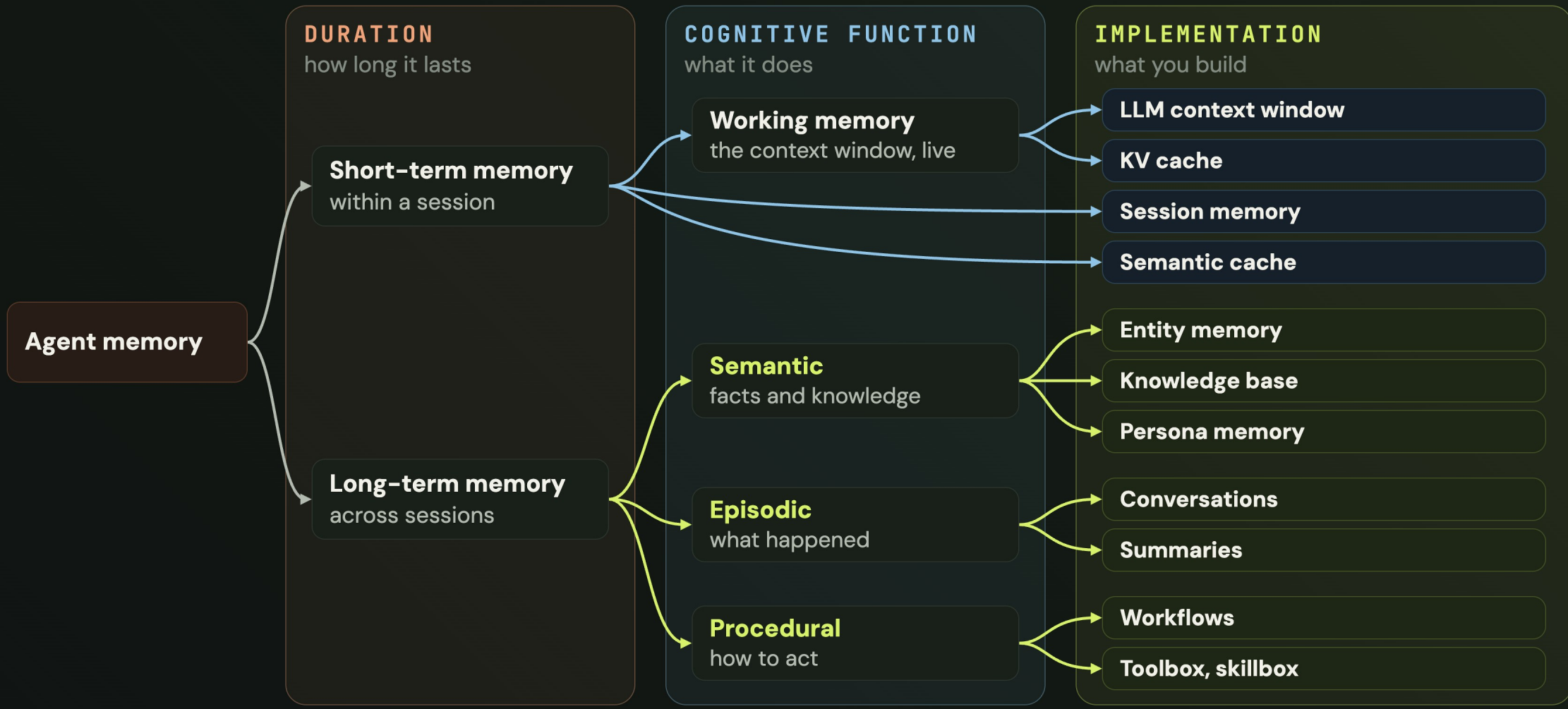
memories · episodes · tools, skills

READ	a query: filter, search, rerank, budget
WRITE	extraction after a turn, or a save
LIVES	rows with a type, scope, TTL, source
COST	storage and a query; yours to govern

Memory engineering decides what belongs where: extract into the database, recall into the window, train what has earned it.

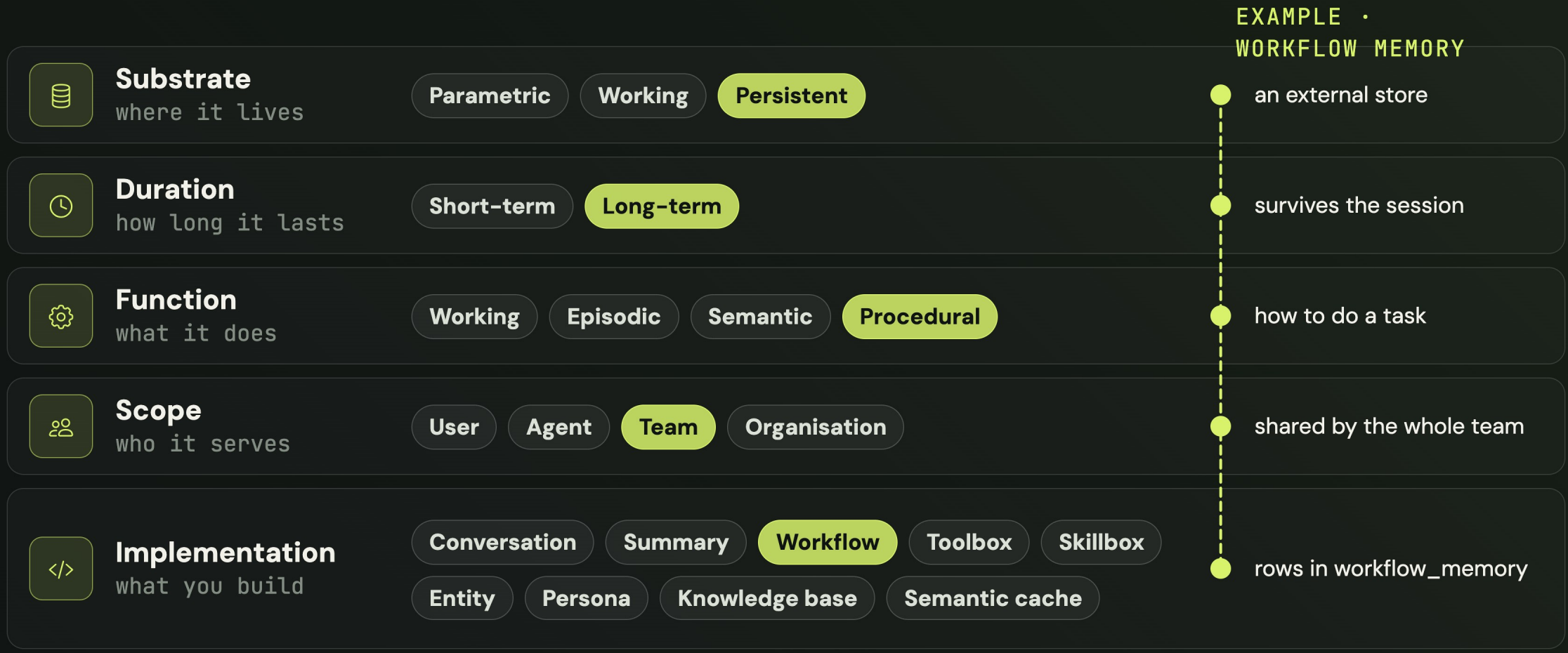
01 · WHAT IS AGENT MEMORY

Agent memory is not one thing



01 · WHAT IS AGENT MEMORY

One taxonomy, five lenses



Four lenses describe any memory. The fifth is what you actually build, and each type has an answer on the other four.

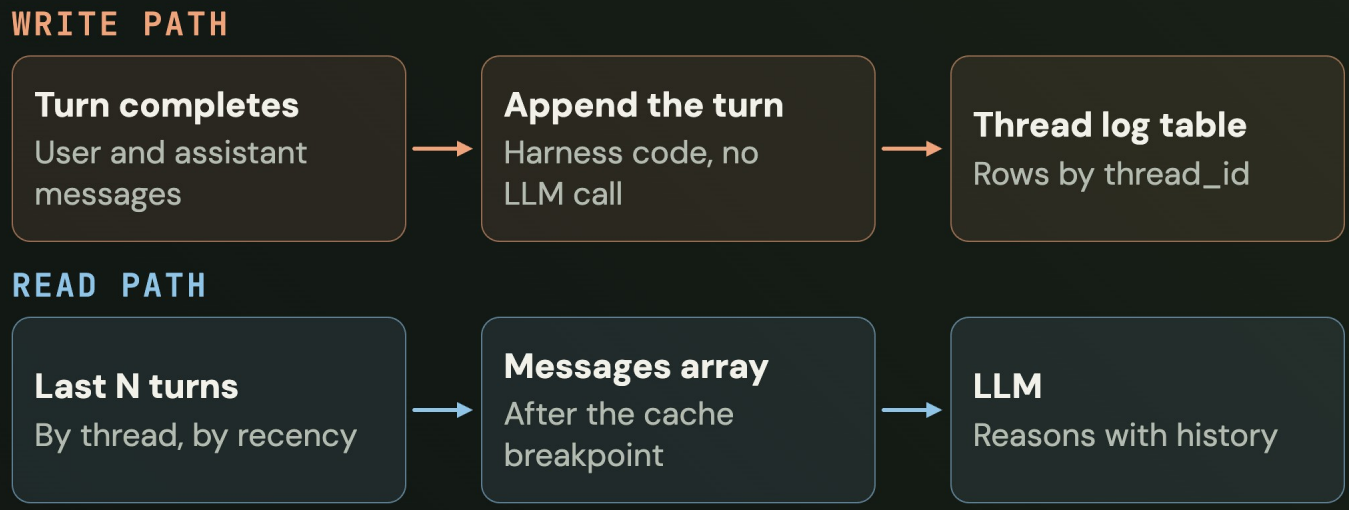
01 · MEMORY TYPES · EPISODIC, PER THREAD

Conversational memory

The raw turns of a dialogue, stored per thread so the agent can pick up where it left off. The simplest type to build, and the fastest to grow.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "conversational",
 "thread_id": "t_8812",
 "user_id": "u_314",
 "role": "user",
 "content": "Book the 9am slot again",
 "created_at": "2026-10-02T17:05Z"
}
```



STORED AS A ROW `thread_messages`

id	thread_id	user_id	role	content	created_at
msg_9f2	t_8812	u_314	user	Book the 9am slot again	2026-10-02 17:05

01 · MEMORY TYPES · IN ACTION

Conversational memory in action

SCENARIO

Maya, booking a dental appointment

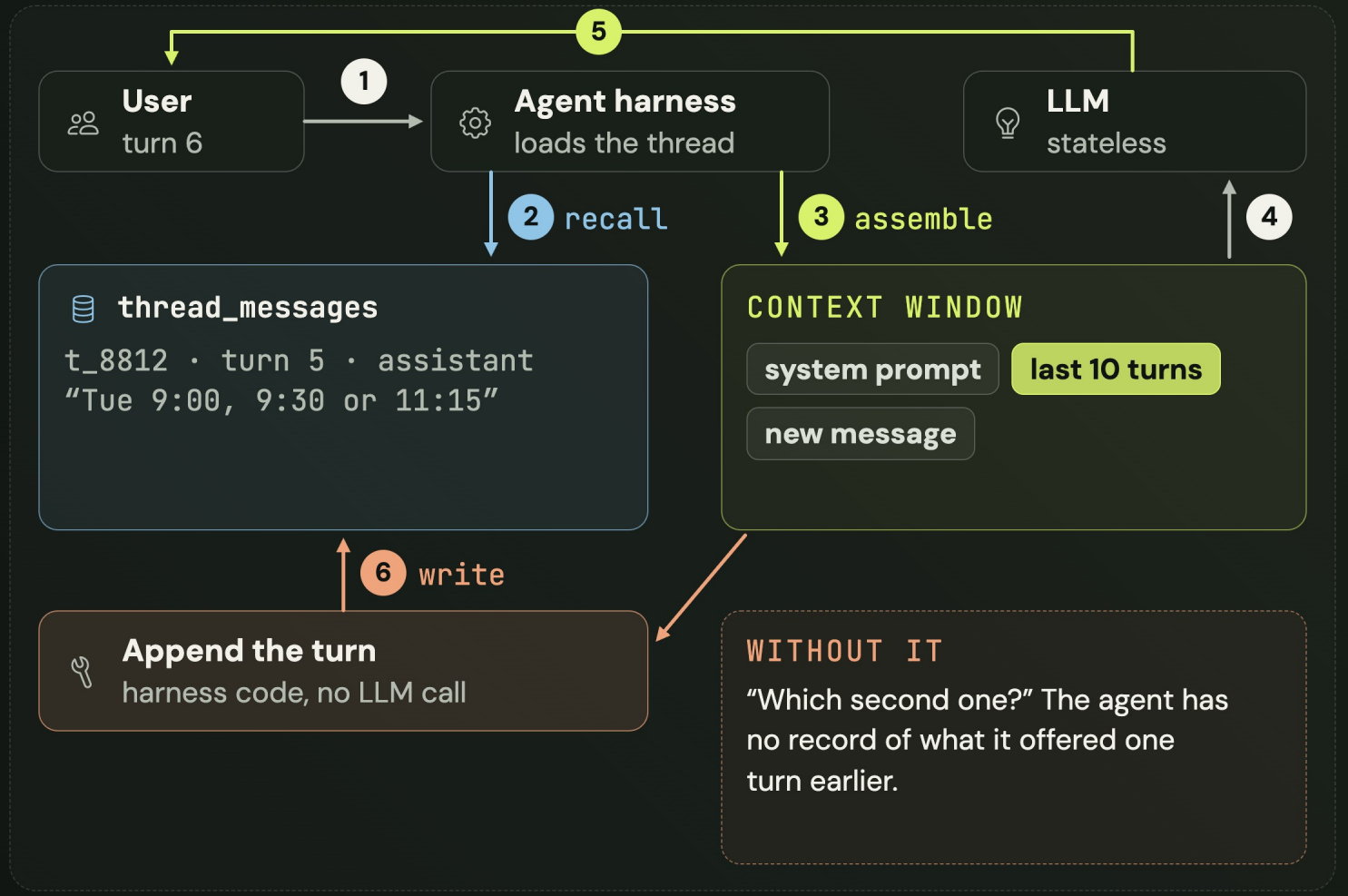
Turn 6 of one thread. Earlier the agent offered three Tuesday slots.

user

Book the second one.

agent

Booked: **Tuesday 9:30** with Dr Okafor. You'll get a reminder the day before.



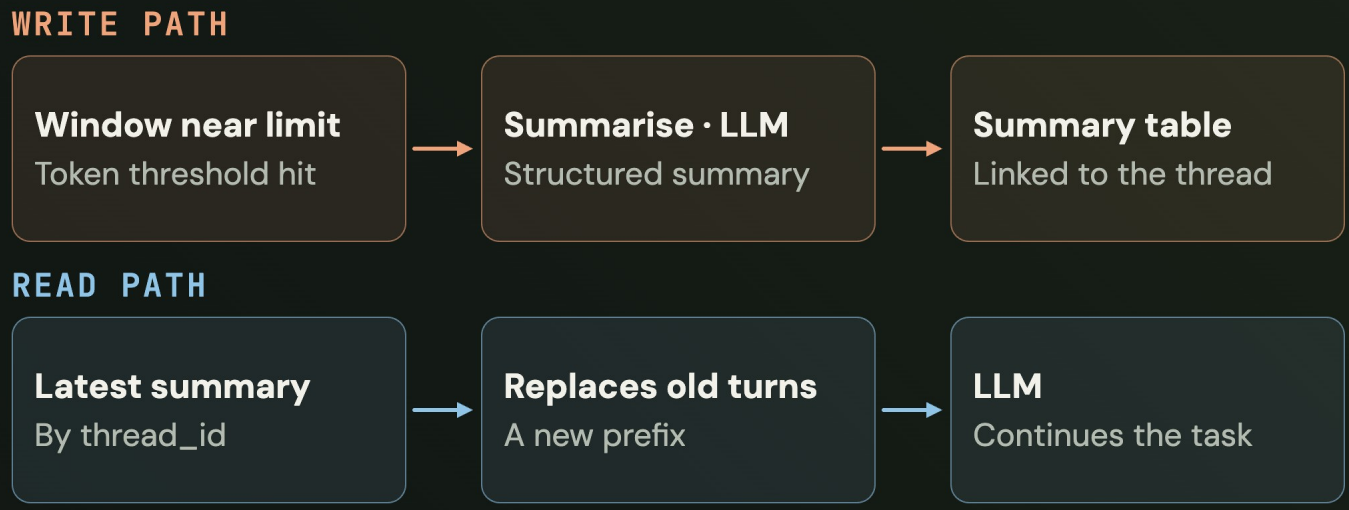
01 · MEMORY TYPES · EPISODIC, COMPACTION

Summary memory

A compressed version of older turns that replaces them in the window. It keeps long sessions inside the limit, but anything the summary drops is gone unless it was stored elsewhere.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "summary",
 "thread_id": "t_8812",
 "covers_turns": [1, 40],
 "summary": "Migrating billing...",
 "open_items": ["confirm VAT"],
 "created_at": "2026-10-02T17:40Z"
}
```



STORED AS A ROW `thread_summaries`

id	thread_id	covers_turns	summary	open_items	created_at
sum_204	t_8812	1 to 40	Migrating billing...	confirm VAT	2026-10-02 17:40

01 · MEMORY TYPES · IN ACTION

Summary memory in action

SCENARIO

Dev, a long billing migration session

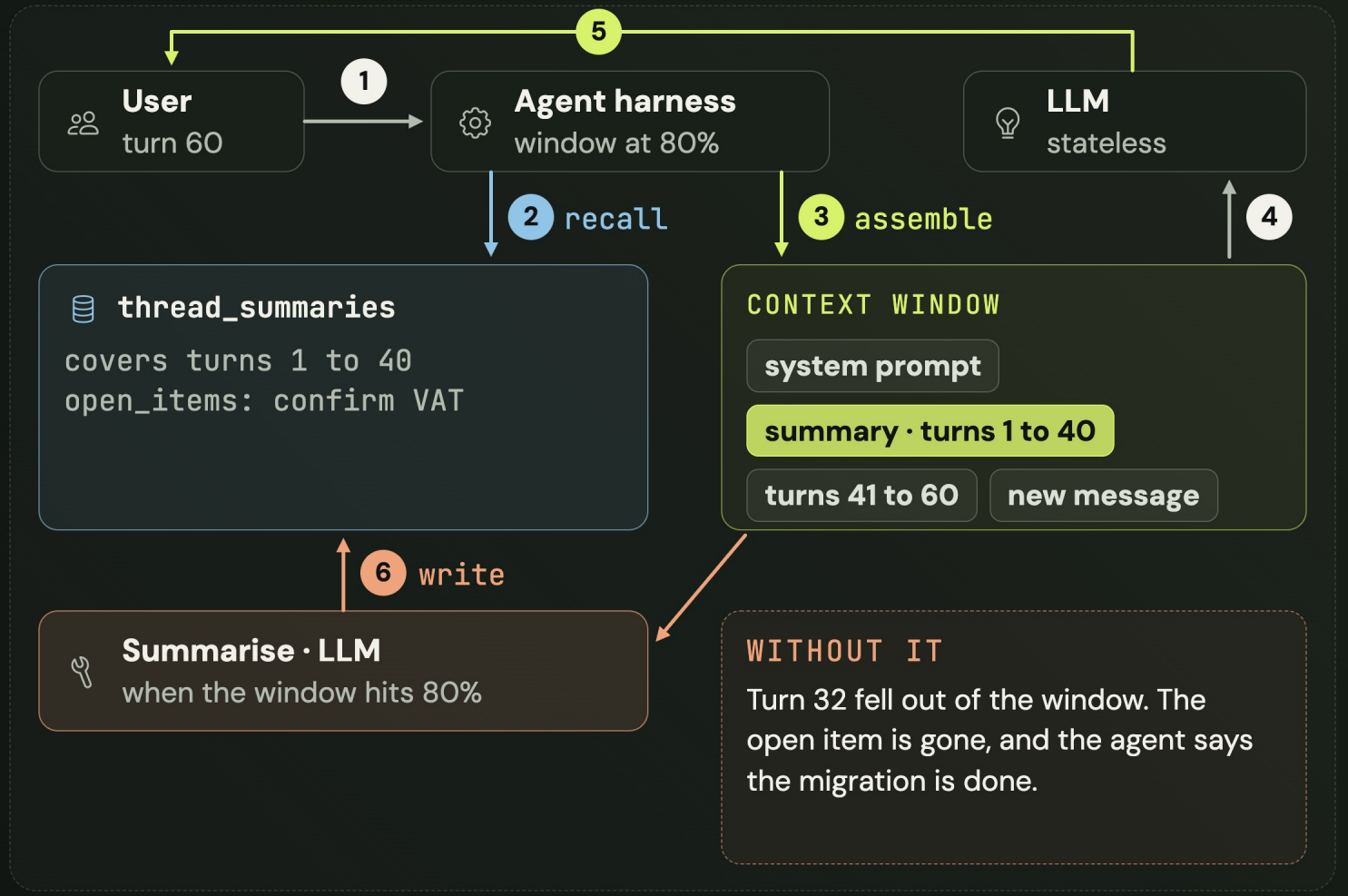
Turn 60. The window passed its threshold at turn 40, so turns 1 to 40 were summarised.

user

Where are we on the VAT question?

agent

Still open: **confirm VAT treatment for EU invoices**. We parked it at turn 32, waiting on finance.



WITHOUT IT

Turn 32 fell out of the window. The open item is gone, and the agent says the migration is done.

01 · MEMORY TYPES · PROCEDURAL

Workflow memory

What happened when the agent ran a task: the steps, the outcome and why it worked or failed. Retrieved before planning so the agent does not repeat a failed path.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "workflow",
 "task": "match invoice to PO",
 "steps": ["parse", "lookup",
"approve"],
 "outcome": "failed",
 "lesson": "match PO by number",
 "embedding": [0.12, -0.07, ...]
}
```

STORED AS A ROW `workflow_memory`

id	task	steps	outcome	lesson	embedding
wf_771	match invoice to PO	parse, lookup, approve	failed	match PO by number	[0.12, -0.07, …]

01 · MEMORY TYPES · IN ACTION

Workflow memory in action

SCENARIO

Priya, accounts payable operations

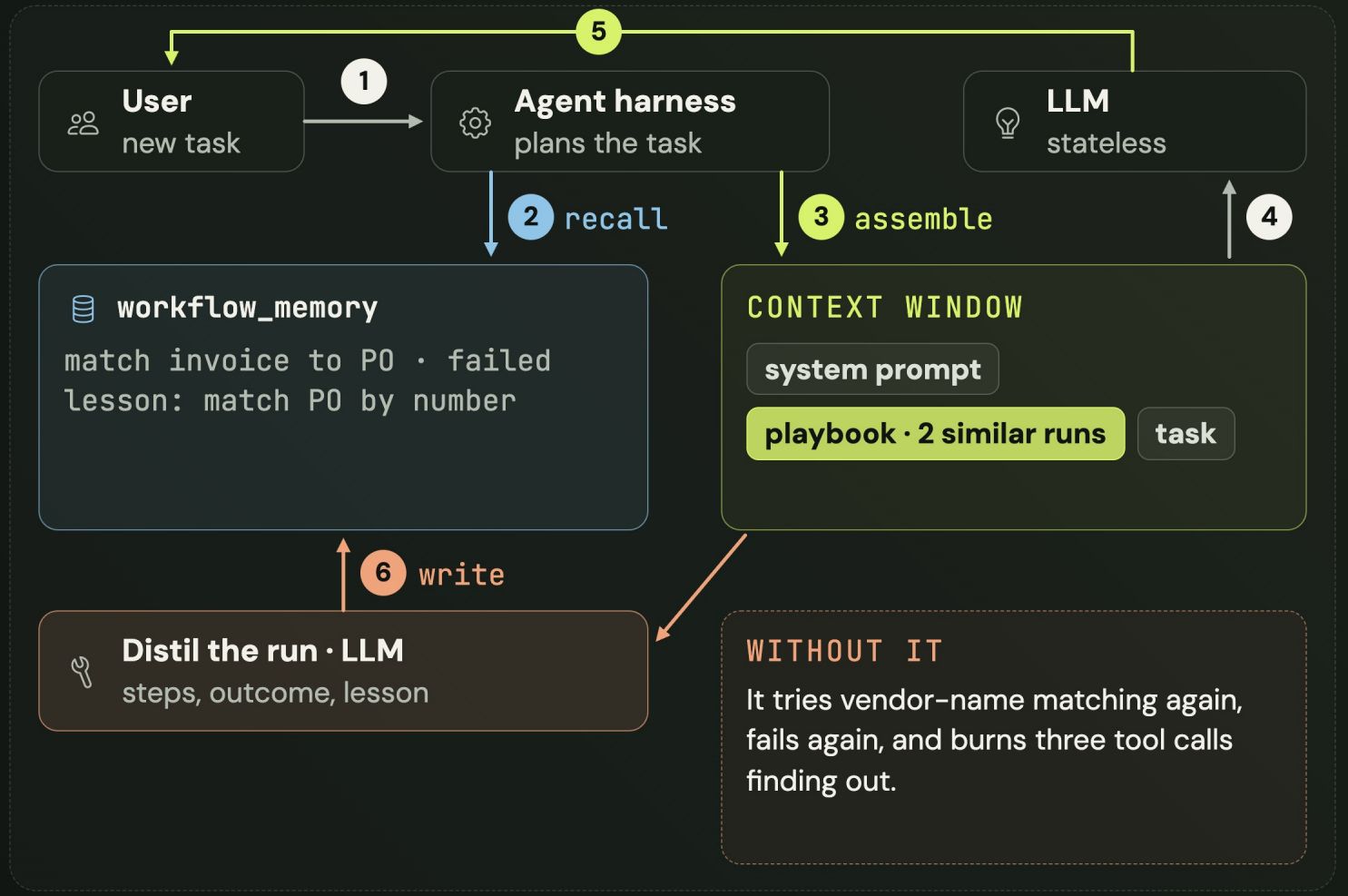
The agent has matched invoices for this supplier before. Two runs failed.

user

Match INV-2207 to its purchase order.

agent

Matched to **PO-5531 by PO number**. Vendor-name matching failed twice for this supplier, so I skipped it.



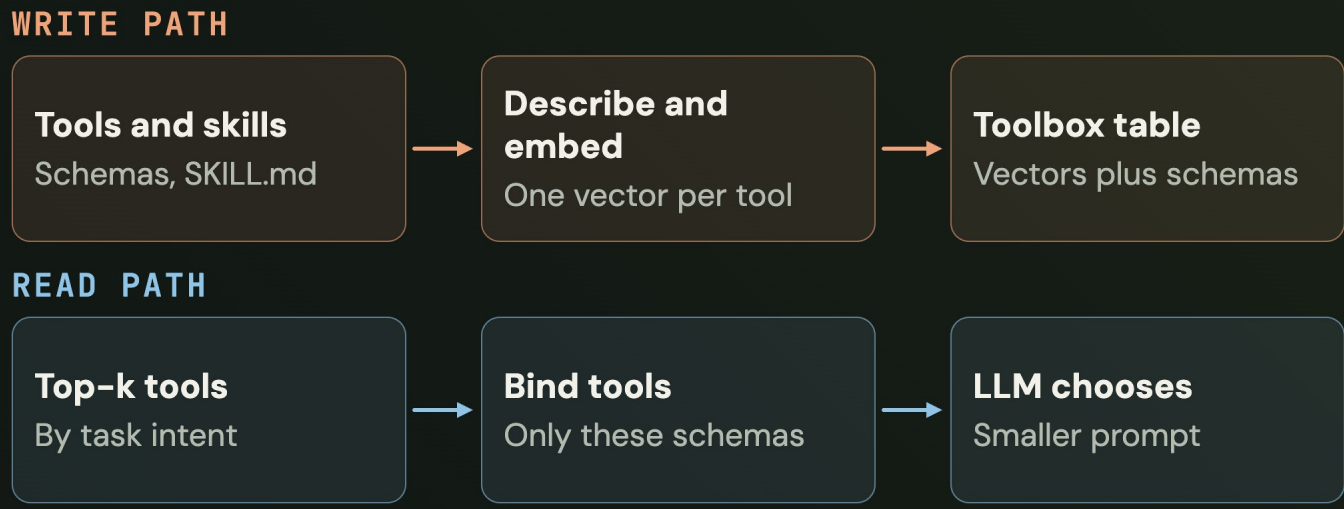
01 · MEMORY TYPES · PROCEDURAL

Toolbox and skillbox memory

Tool and skill definitions stored outside the window and retrieved by relevance, so the agent loads the few it needs instead of every schema on every turn.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "tool",
 "name": "get_invoice",
 "description": "Fetch an invoice",
 "schema": {"invoice_id": "string"},
 "embedding": [0.31, 0.02, ...],
 "success_rate": 0.97
}
```



STORED AS A ROW `toolbox`

id	name	description	schema	embedding	success_rate
tool_118	get_invoice	Fetch an invoice	{invoice_id}	[0.31, 0.02, …]	0.97

01 · MEMORY TYPES · IN ACTION

Toolbox memory in action

SCENARIO

Sam, a customer on the support chat

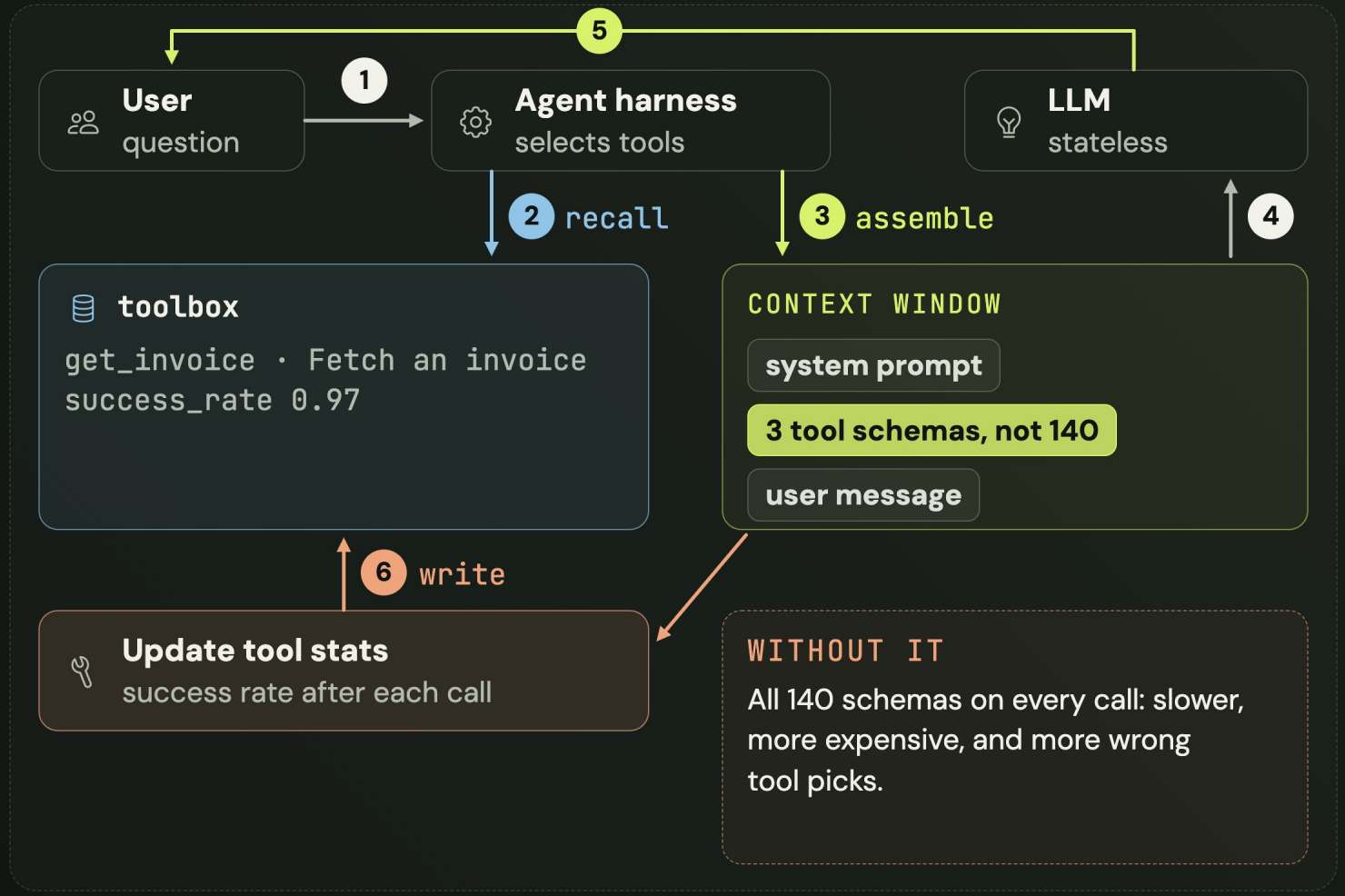
The support agent has 140 tools. Loading every schema costs about 30k tokens a call.

user

Can I get a copy of my March invoice?

agent

Here it is: **invoice A-3391** for March, also sent to your email.



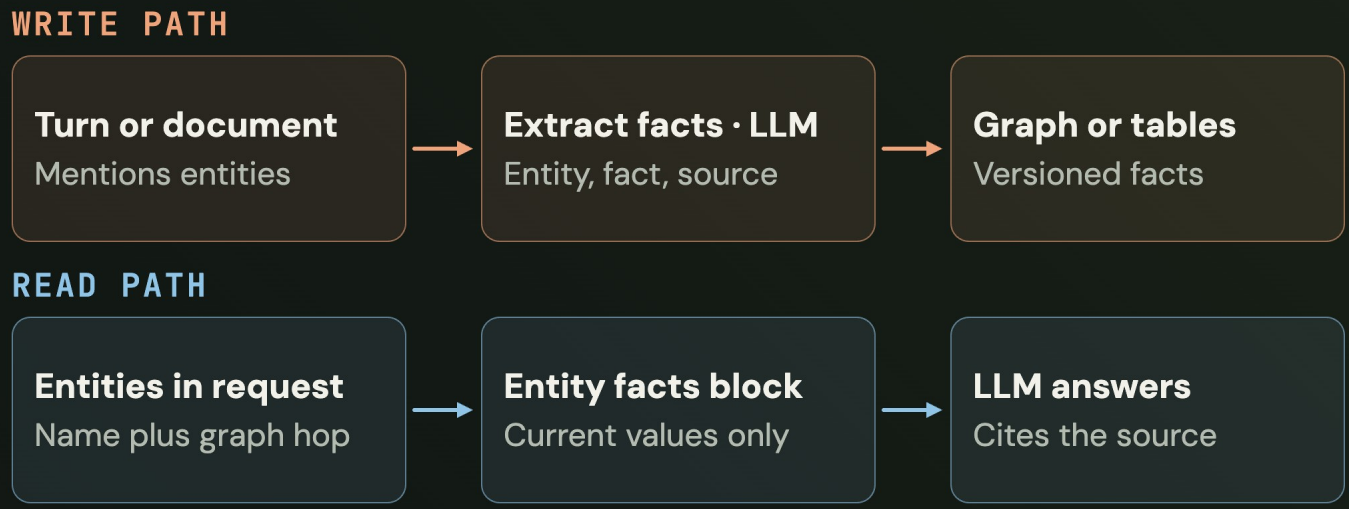
01 · MEMORY TYPES · SEMANTIC

Entity memory

Facts about the people, organisations and things the agent meets, kept current as they change and linked back to their source.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "entity",
 "name": "Acme Ltd",
 "kind": "customer",
 "facts": {"plan": "enterprise"},
 "valid_from": "2026-09-01",
 "source": "ticket #1042"
}
```



STORED AS A ROW `entity_facts`

id	entity	kind	fact	valid_from	valid_to	source
ent_52	Acme Ltd	customer	plan = enterprise	2026-09-01	null	ticket #1042

01 · MEMORY TYPES · IN ACTION

Entity memory in action

SCENARIO

Tom, an account manager

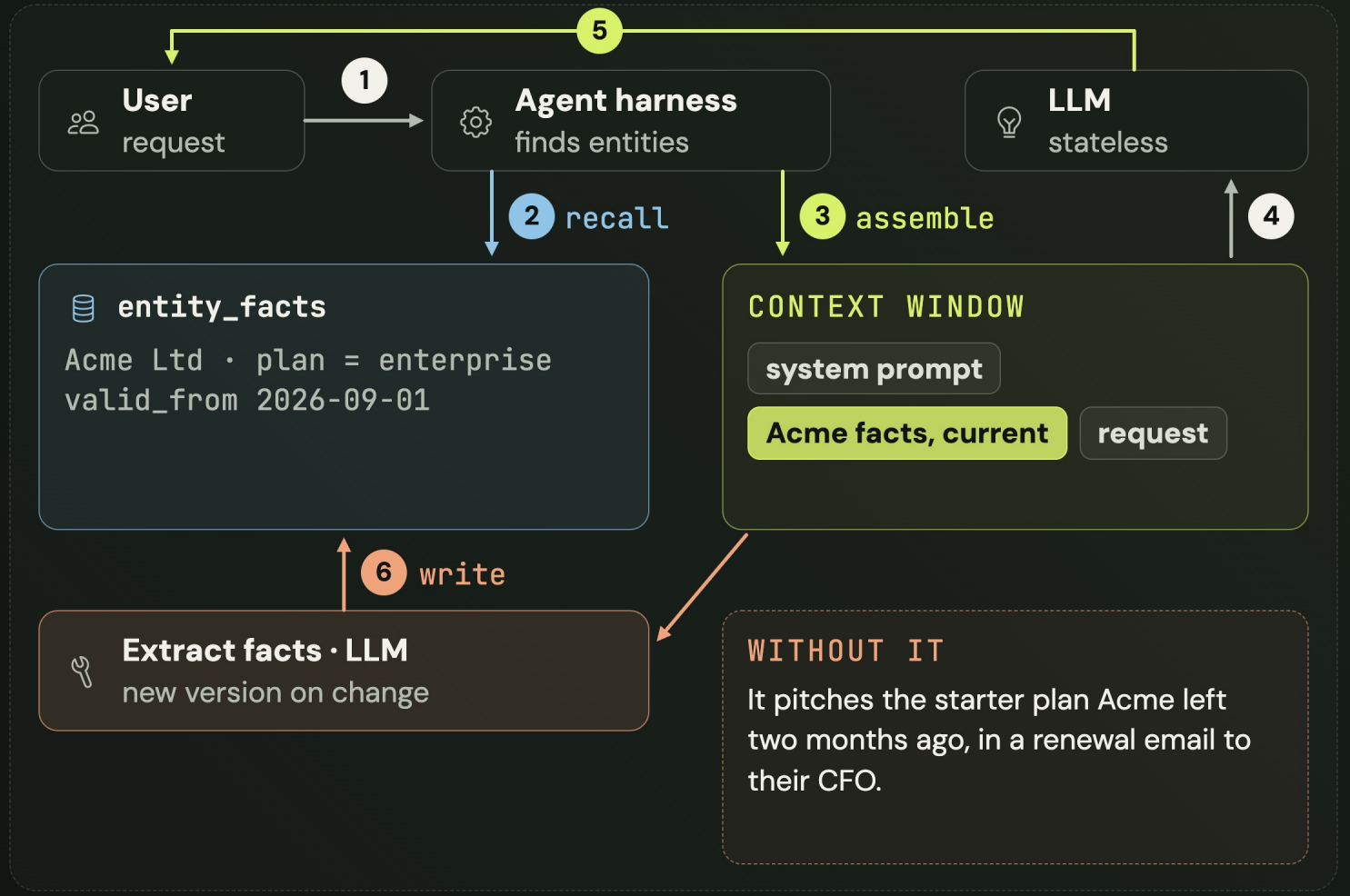
Acme Ltd moved from the starter plan to enterprise on 1 September.

user

Draft a renewal note to Acme.

agent

Draft ready. It references their **enterprise plan since 1 Sept** and the open ticket #1042.



WITHOUT IT

It pitches the starter plan Acme left two months ago, in a renewal email to their CFO.

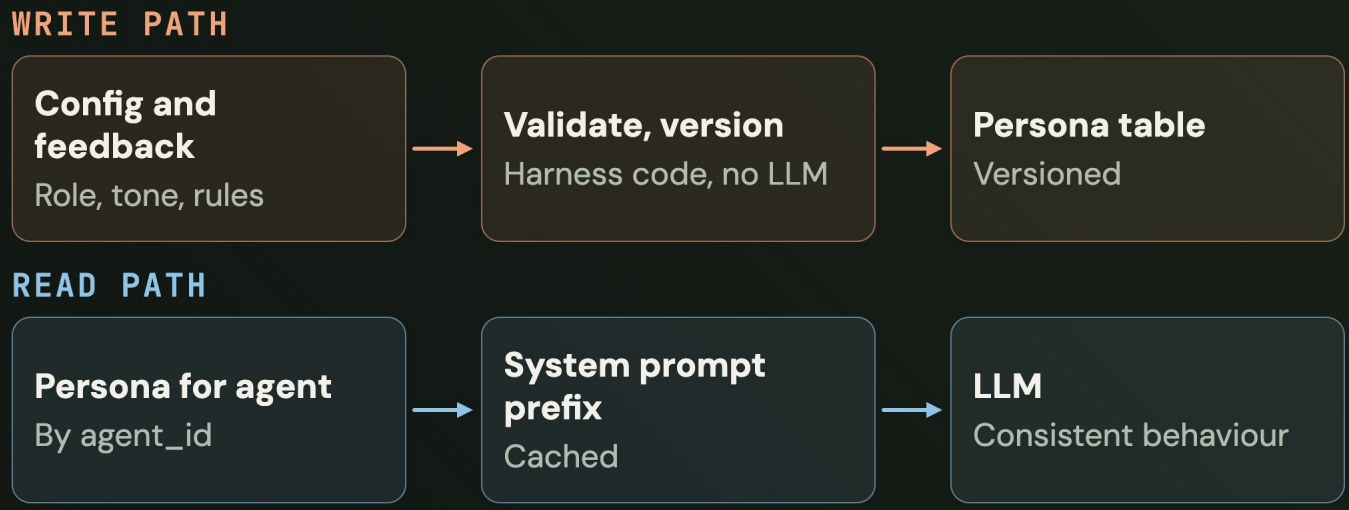
01 · MEMORY TYPES · SEMANTIC

Persona memory

The agent's role, tone, goals and standing rules. Stable by design, which makes it a natural fit for the cached prefix of the prompt.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "persona",
 "agent_id": "support_copilot",
 "role": "Tier 1 support agent",
 "tone": "concise, warm",
 "rules": ["never promise refunds"],
 "version": 4
}
```



STORED AS A ROW `personas`

agent_id	version	role	tone	rules	updated_at
support_copilot	4	Tier 1 support agent	concise, warm	never promise refunds	2026-10-01

01 · MEMORY TYPES · IN ACTION

Persona memory in action

SCENARIO

A frustrated customer, support chat

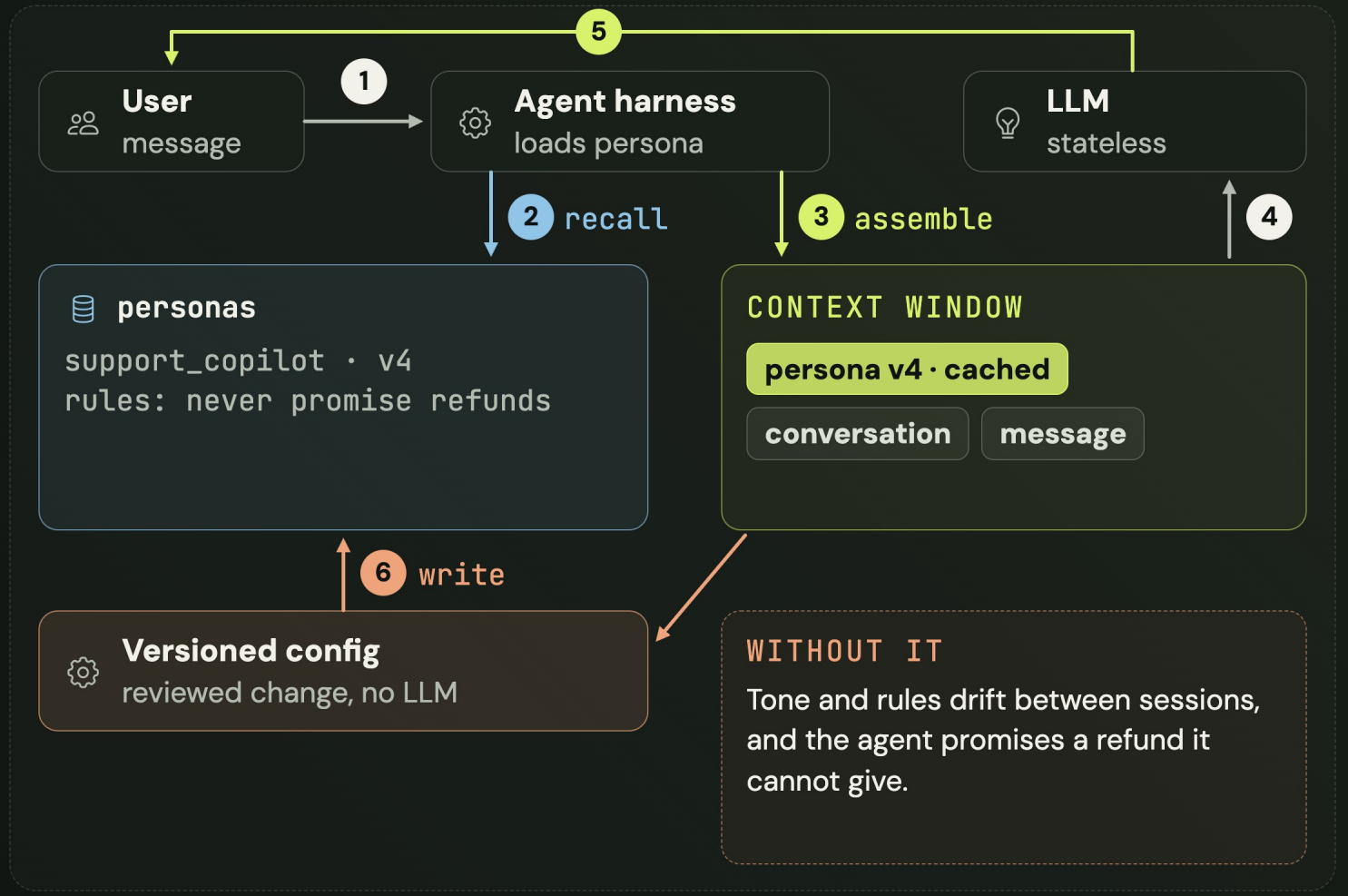
The support copilot runs persona v4: concise, warm, never promise refunds.

user

Just refund me now.

agent

I'm sorry about this. **I can't promise a refund**, but I've raised it with billing and you'll hear back today.



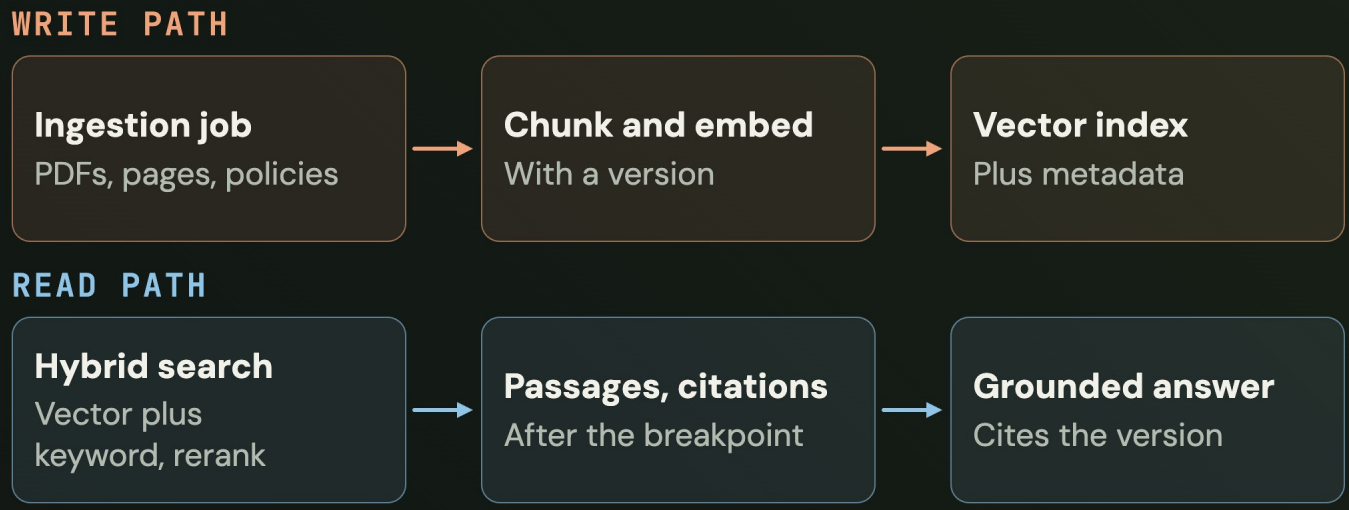
01 · MEMORY TYPES · SEMANTIC

Knowledge base

Documents, policies and reference material the agent grounds on. Not the agent's own experience, but it has to be current and authoritative, as Air Canada learned.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "knowledge",
 "doc_id": "policy_bereavement",
 "chunk": "Requests must be...",
 "version": "2026-03",
 "source": "policies/travel.pdf",
 "embedding": [0.08, 0.44, ...]
}
```



STORED AS A ROW `kb_chunks`

chunk_id	doc_id	version	chunk	source	embedding
kb_3310	policy_bereavement	2026-03	Requests must be...	policies/travel.pdf	[0.08, 0.44, …]

01 · MEMORY TYPES · IN ACTION

Knowledge base in action

SCENARIO

Leo, an airline customer

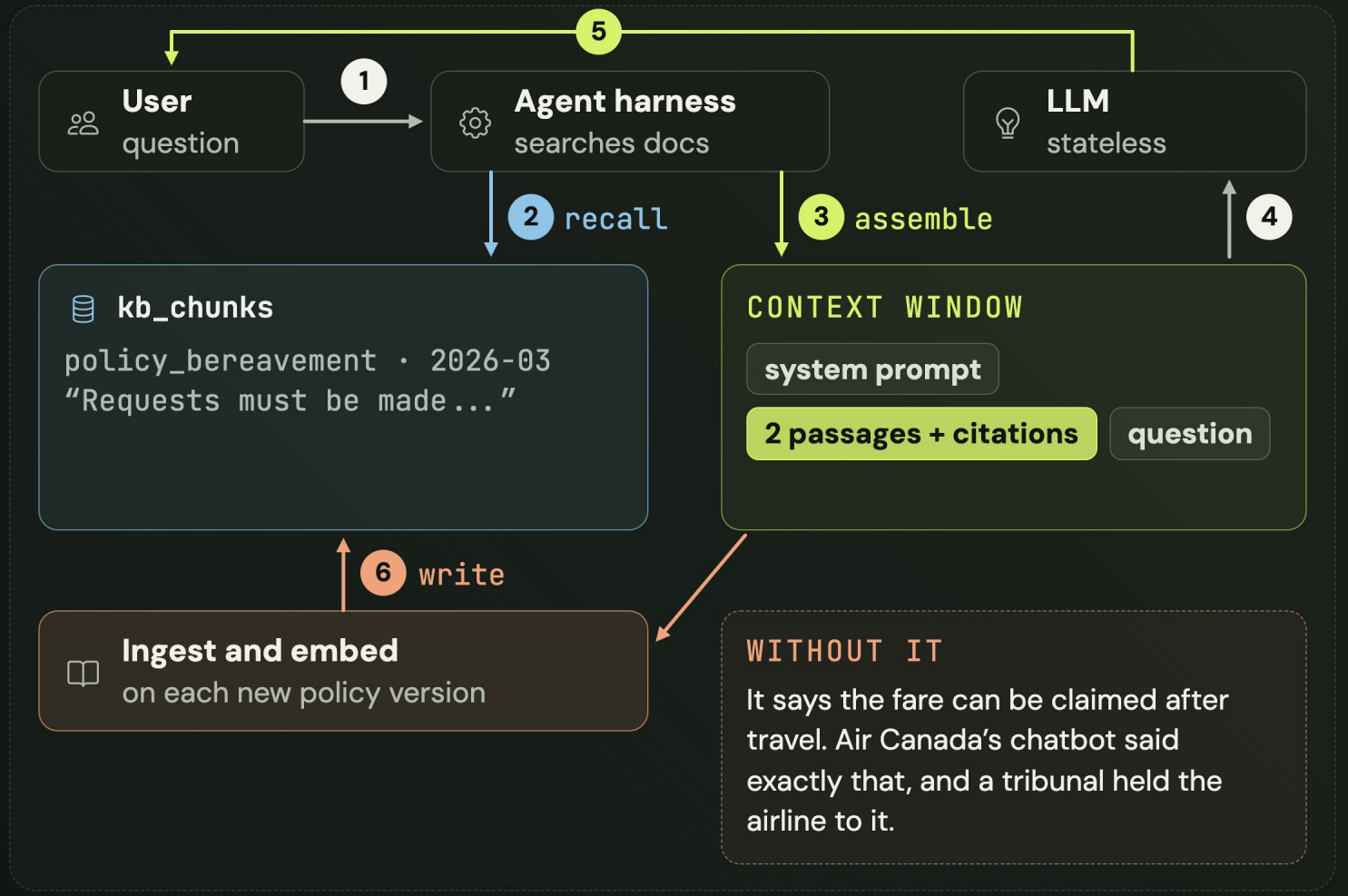
The bereavement fare policy changed in March 2026. Old and new versions are both indexed.

user

Can I claim the bereavement fare after I fly?

agent

Under the **March 2026 policy**, requests must be made before travel. Here is the section.



WITHOUT IT

It says the fare can be claimed after travel. Air Canada’s chatbot said exactly that, and a tribunal held the airline to it.

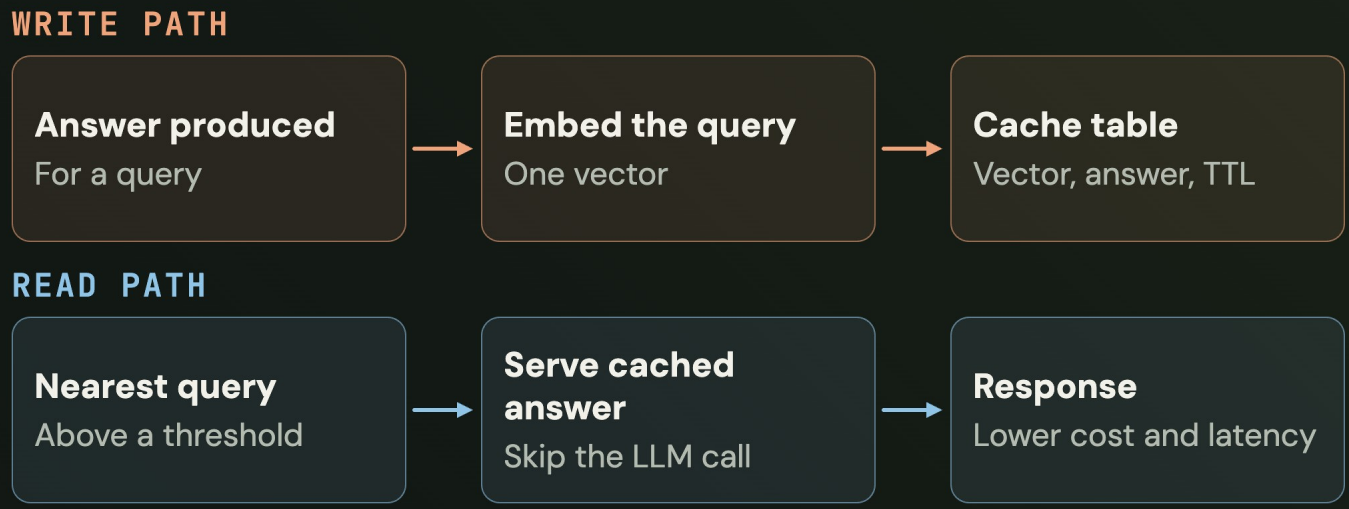
01 · MEMORY TYPES · SHORT-TERM, REUSE

Semantic cache

Past answers stored against the meaning of the question. When a new query is close enough, the cached answer is returned and the model call is skipped.

MEMORY UNIT · ILLUSTRATIVE

```
{
 "type": "semantic_cache",
 "query": "reset my router",
 "embedding": [0.27, -0.13, ...],
 "response": "Hold reset 10s...",
 "ttl_hours": 24,
 "hits": 182
}
```



STORED AS A ROW `semantic_cache`

id	query	embedding	response	ttl_hours	hits
sc_901	reset my router	[0.27, -0.13, …]	Hold reset 10s...	24	182

01 · MEMORY TYPES · IN ACTION

Semantic cache in action

SCENARIO

An ISP support bot, Monday morning

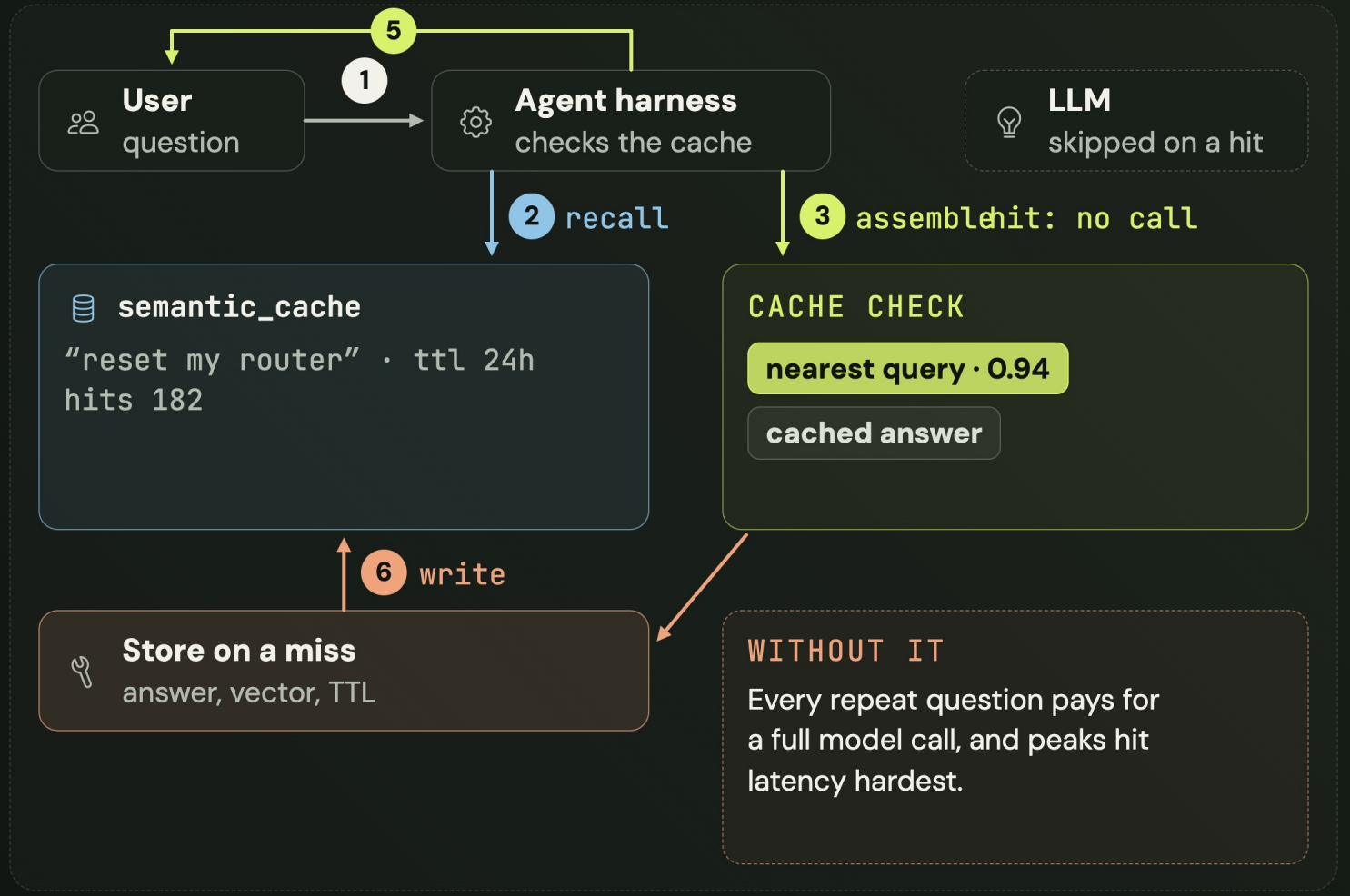
Thousands of customers ask the same router question in different words.

user

My router's frozen, how do I reset it?

agent

Hold the reset button for 10 seconds until the light blinks. **Served from cache**, no model call.



MODULE 02

The agent memory landscape

Every tier of the stack now ships memory: personal agents, frontier labs, clouds and startups.

02 · THE RISE OF PERSONAL AGENTS

In 2026 every major lab shipped a personal agent

Each runs in the background on a cloud computer, works across your apps, and keeps learning about you between conversations.

May 19 2026

Gemini Spark

Announced at I/O: a 24/7 agent using Personal Intelligence.

Aug 11 2026

Grok Bot

xAI launches role-based Bots, each with its own memory.

Sept 8 2026

Meta Muse

A personal agent that keeps an editable Memory.md file.

Sept 29 2026

OpenAI dots

Starts from ChatGPT memory, then keeps its own notes.

Oct 21 2026

Claude Duet

Research preview that uses and adds to your Claude memory.

Sources: Google I/O, 19 May 2026; xAI, Meta, OpenAI and Anthropic launch posts, as compiled by MemoryLake, 26 October 2026.

02 · THE RISE OF PERSONAL AGENTS

Four personal agents, four memory designs

	Claude Duet	OpenAI dots	Grok Bot	Meta Muse
Starts from	Your Claude memory, skills, connectors	Relevant ChatGPT memory	Nothing; learns as it works	Nothing; learns as it works
Where it lives	Your Claude account memory	The dot's own notes	Each Bot's own memory	Memories, incl. Memory.md
Scope	Shared with chat and Cowork	Same dot across ChatGPT, Slack, Teams	Per Bot; team memory for Team Bots	Within Muse
Forgetting	Claude memory settings	Deleting a dot removes its context	Deleting a Bot erases its memory	A forget skill, best effort

Source: vendor help pages for each agent, as compiled by MemoryLake, 26 October 2026. Check current docs before relying on any row.

02 · THE RISE OF PERSONAL AGENTS

How assistant memory improved in 2026

01 · BACKGROUND CONSOLIDATION

Memory is written between conversations

ChatGPT's Dreaming V3, dreaming on Claude Managed Agents and OpenClaw's dreaming all consolidate offline.

02 · INSPECTABLE MEMORY

Users can read and correct it

ChatGPT's memory summary page, Claude's categorised entries and Muse's Memory.md.

03 · TIME AWARENESS

Plans become history

OpenAI says the 2026 system tells active plans from completed events, so a past trip stops being a current plan.

04 · SCOPED MEMORY

Memory follows a role or a channel

Per-Bot memory, team versus personal notes, and MEMORY.md kept out of group chats in OpenClaw.

05 · CARRIED-OVER MEMORY

No cold start

Duet and dots begin with what the chatbot already knows about you.

06 · STILL UNSOLVED

Forgetting

Deleting a message or turning a setting off does not always remove what an agent learned from it.

02 · HOW MEMORY WORKS AT ANTHROPIC

One profile for people, files for developers

CLAUDE APP · CONSUMER

Chat
Reads and updates entries as you talk

Cowork
Same memory since August 2026

Claude account memory
Categorised entries you can view, edit or delete

Past chats
Searched when a detail beats a memory

Claude Duet
Uses and adds to the same memory

CLAUDE CODE AND CLAUDE PLATFORM · DEVELOPERS

Claude Code

CLAUDE.md
You write it; team rules

MEMORY.md
Claude writes; first 200 lines load

→ **Topic files**
Read on demand

Claude Platform

Memory tool
Client-side files on your storage

Compaction and context editing
Trim the window

Managed Agents memory
Versioned stores, plus dreaming

02 · HOW MEMORY WORKS AT ANTHROPIC

Claude’s memory went from a daily summary to entries

Sep 2025

Team and Enterprise
Memory for work accounts

Oct 2025

Pro and Max
Search chats and generate memory

Mar 2026

All users
Free plan, plus a memory import tool

Jul 2026

Categorised entries
Replace the daily memory summary

Aug 2026

Cowork
One memory across chat and Cowork

Oct 2026

Claude Duet
Uses and adds to your memory

The July 2026 change matters most for engineers: a rolling daily summary became discrete entries that Claude reads, writes and updates during a conversation, so a correction takes effect in the next chat, not the next day.

Sources: MacRumors, 23 Oct 2025; NewsBytes, 3 Mar 2026; Anthropic release notes via MemoryLake, July 2026; TechCrunch, 25 Aug 2026; Anthropic, 21 Oct 2026.

02 · HOW MEMORY WORKS AT ANTHROPIC

Five memory primitives on the Claude Platform

Primitive	What it does	Who holds the memory
Memory tool	Claude asks for file operations on a /memories directory; your handler runs them	You, client-side
Context editing	Clears stale tool results near the limit; 84% fewer tokens in Anthropic's 100-turn eval	Nobody: it trims the window
Server-side compaction	Summarises older context on the server near the limit	Anthropic, as a summary
Managed Agents memory	Workspace-scoped stores of text files, exportable, with version history (public beta, Apr 2026)	Anthropic, versioned
Dreaming	Scheduled review of recent sessions that updates Managed Agents memory (research preview, May 2026)	Anthropic, in the background

Sources: Anthropic, Managing context on the Claude Developer Platform, Sept 2025; Claude docs, Using agent memory; Anthropic Code with Claude, 6 May 2026.

02 · HOW MEMORY WORKS AT OPENAI

Synthesis for people, files and sessions for developers

CHATGPT · CONSUMER

Saved memories
Explicit items, still available

Chat history
Past conversations

Dreaming V3 · background synthesis
Reads many past chats offline and revises what it knows

Memory summary
You read, correct and steer it

dots
Start from ChatGPT memory, keep own notes

CODEX AND THE API · DEVELOPERS

Codex

AGENTS.md
Team rules, checked in; read first

Memories
Off by default; consolidated in the background

OpenAI API and Agents SDK

Conversations API
Server-managed state

Compaction
The compact endpoint shortens history

SDK sessions
SQLite, Redis or Conversations

02 · HOW MEMORY WORKS AT OPENAI

ChatGPT moved from saved items to background synthesis

2024

Saved memories
A list you could see and delete

Apr 2025

Chat history
First Dreaming: context beyond saved items

Jan 2026

Better recall
Answers link to the source chats

Jun 2026

Dreaming V3
Summary page, time-aware, 2x capacity

Sept 2026

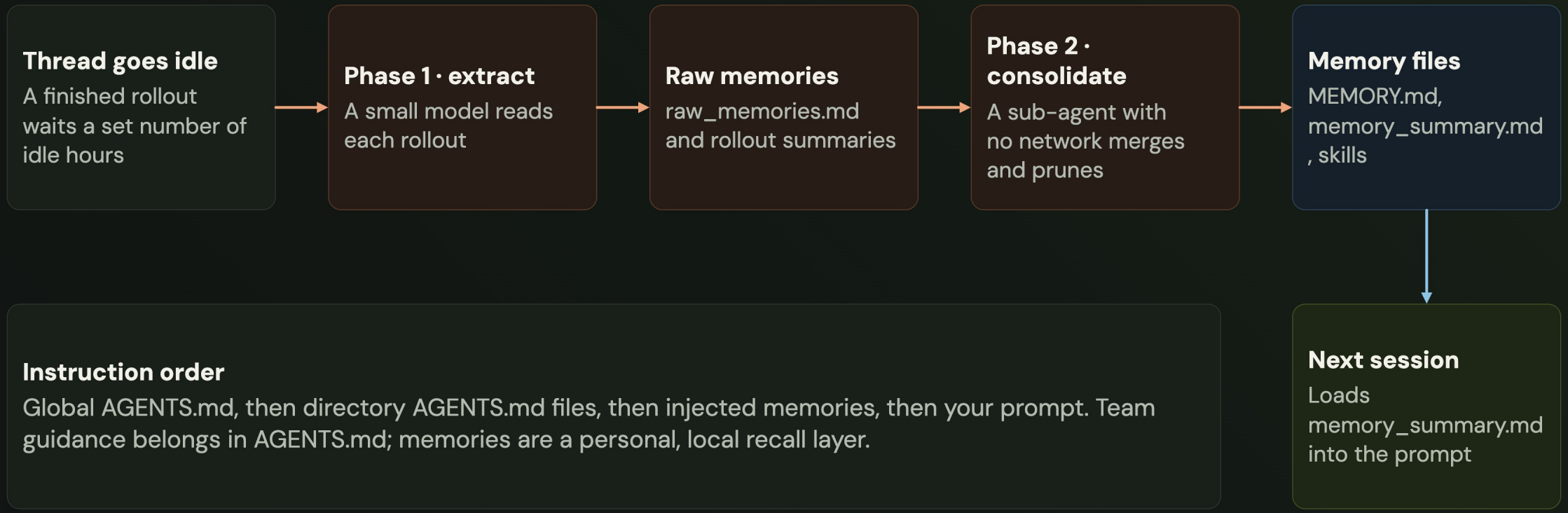
dots
Personal agents inherit ChatGPT memory

OpenAI says the June 2026 architecture cut the compute for background memory by about five times, which is what let it reach free users, while Plus and Pro got twice the capacity.

Sources: OpenAI, Dreaming: Better memory for a more helpful ChatGPT, 4 June 2026; ChatGPT release notes; Tom's Guide, Jan 2026; MemoryLake, Oct 2026.

02 · HOW MEMORY WORKS AT OPENAI

Codex consolidates memory in two background phases



Sources: OpenAI Codex docs (Memories, AGENTS.md) as described by Mem0 and Daniel Vaughan, April to June 2026. Memories reached the EEA, UK and Switzerland in June 2026, off by default there.

02 · HOW MEMORY WORKS AT OPENAI

OpenAI’s developer stack: state, compaction, sessions

Primitive	What it does	Who holds the memory
Conversations API	Stores conversation state on OpenAI’s side, referenced by ID	OpenAI
responses.compact	Compacts stored history into a shorter form	OpenAI, as compacted items
Agents SDK sessions	Client-side history: SQLite, Redis, SQLAlchemy, Dapr, or Conversations-backed	You, or OpenAI
Compaction session	Wraps any session and calls responses.compact when history grows	Your session store
Sandbox memory (Apr 2026)	File-based, task-level memory that survives across sessions	Your sandbox

Sources: OpenAI Agents SDK docs, Sessions and OpenAIResponsesCompactionSession; OpenAI Agents SDK update, 15 April 2026.

02 · FRONTIER LABS SIDE BY SIDE

Anthropic and OpenAI made different bets

	Anthropic	OpenAI
Consumer memory written	During the conversation, as entries	In the background, by synthesis
What users see	Categorised entries to edit	A memory summary to steer
Coding agent	Claude Code auto memory, on by default	Codex Memories, off by default
Team sharing	CLAUDE.md, checked in	AGENTS.md, checked in
Developer storage	Memory tool (yours) or Managed Agents stores	Sessions (yours) or Conversations API
Background consolidation	Dreaming on Managed Agents	Dreaming in ChatGPT; Codex phase 2

02 · THE AGENT MEMORY LANDSCAPE

Memory at the hyperscalers

GOOGLE · VERTEX AI MEMORY BANK

- Generates memories from sessions in the background
- Scoped to an identity; retrieved by similarity
- Configurable topics, TTLs and revisions

AWS · BEDROCK AGENTCORE MEMORY

- Short-term raw events per session
- Long-term strategies: semantic, summary, preference, episodic, custom
- Per-actor isolation and namespaces

MICROSOFT · FOUNDRY MEMORY

- Managed memory stores
- User profile, chat summary and procedural memory
- Store-level TTLs

Managed memory is now a line item in every cloud. Ask about ownership, portability and isolation. Check current features before choosing.

02 · THE AGENT MEMORY LANDSCAPE

Memory at the startups

Mem0

A memory layer API: extract, consolidate and retrieve.

Zep

A temporal knowledge graph for facts that change over time.

Letta

Stateful agents, from the team behind MemGPT.

Supermemory

A memory API with plugins across tools.

Cognee

Graph and vector memory pipelines.

Hindsight

Agent memory from Vectorize: retain, recall, reflect.

Plus the database tier (Oracle, MongoDB, Redis and others) and frameworks such as LangGraph, Letta and MemoRizz.

02 · THE AGENT MEMORY LANDSCAPE

Different names, one pattern

01
Events in
Raw turns written per session and per actor.

02
Extraction
A model pulls candidates, guided by topics or strategies.

03
Consolidation
Create, update or delete when facts conflict.

04
Scoped storage
By user, agent or namespace, with TTLs.

05
Retrieval
Similarity plus filters, injected into context.

Codex calls step 3 consolidation, ChatGPT and Claude Managed Agents call it dreaming, Memory Bank calls its guidance topics, AgentCore calls it strategies.

02 · MEMORY IS YOUR MOAT · 1 OF 2

Same parts, two agents

	THEIR AGENT Any competitor	VS	YOUR AGENT Your harness, your rows
MODEL	The same frontier models, called by API	=	The same frontier models, called by API
FRAMEWORK AND TOOLS	The same open-source frameworks, MCP servers and APIs	=	The same open-source frameworks, MCP servers and APIs
MEMORY	What theirs learned: different users, corrections and history	≠	What yours learned: your customers, processes, edge cases and corrections

The first two rows are a pip install away for anyone. The last row cannot be downloaded.

THE ARGUMENT

Memory is the only moat. You should own it.

Models are rented and they converge. The memory your agents build from your users, tools and operations is the asset that compounds, so keep it inspectable, portable and under your governance.

General-purpose memory about people will likely sit with the frontier labs and big consumer products. For everyone else the opening is memory built for a domain, and better still memory you can configure per domain and task.

MODULE 03

Implementing memory-aware agents

Context engineering, memory engineering and the reference architecture.

03 · THREE DISCIPLINES

Maximise one call, optimise one session, continue

SCOPE · ONE CALL

Prompt engineering

MAXIMISATION

- **Instruction** — role, examples, schema
- **LLM** — one call, one answer
- **Output** — structured and checked

TECHNIQUES

Role and instruction · Few-shot · Output schema · Compression

SCOPE · ONE SESSION

Context engineering

OPTIMISATION

- **Retriever** — just in time, per step
- **Context window** — stable first, volatile last
- **Compactor** — clear and summarise

TECHNIQUES

JIT retrieval · Stable prefix · Compaction · Tool-result clearing

SCOPE · AGENT LIFETIME

Memory engineering

CONTINUATION

- **Extractor** — after each turn or session
- **Memory store** — typed, scoped, versioned
- **Recall** — next session, next agent

TECHNIQUES

Extraction · Typed stores · Rerank and gates · Forgetting

03 · CONTEXT ENGINEERING

Assembling the window, one call at a time

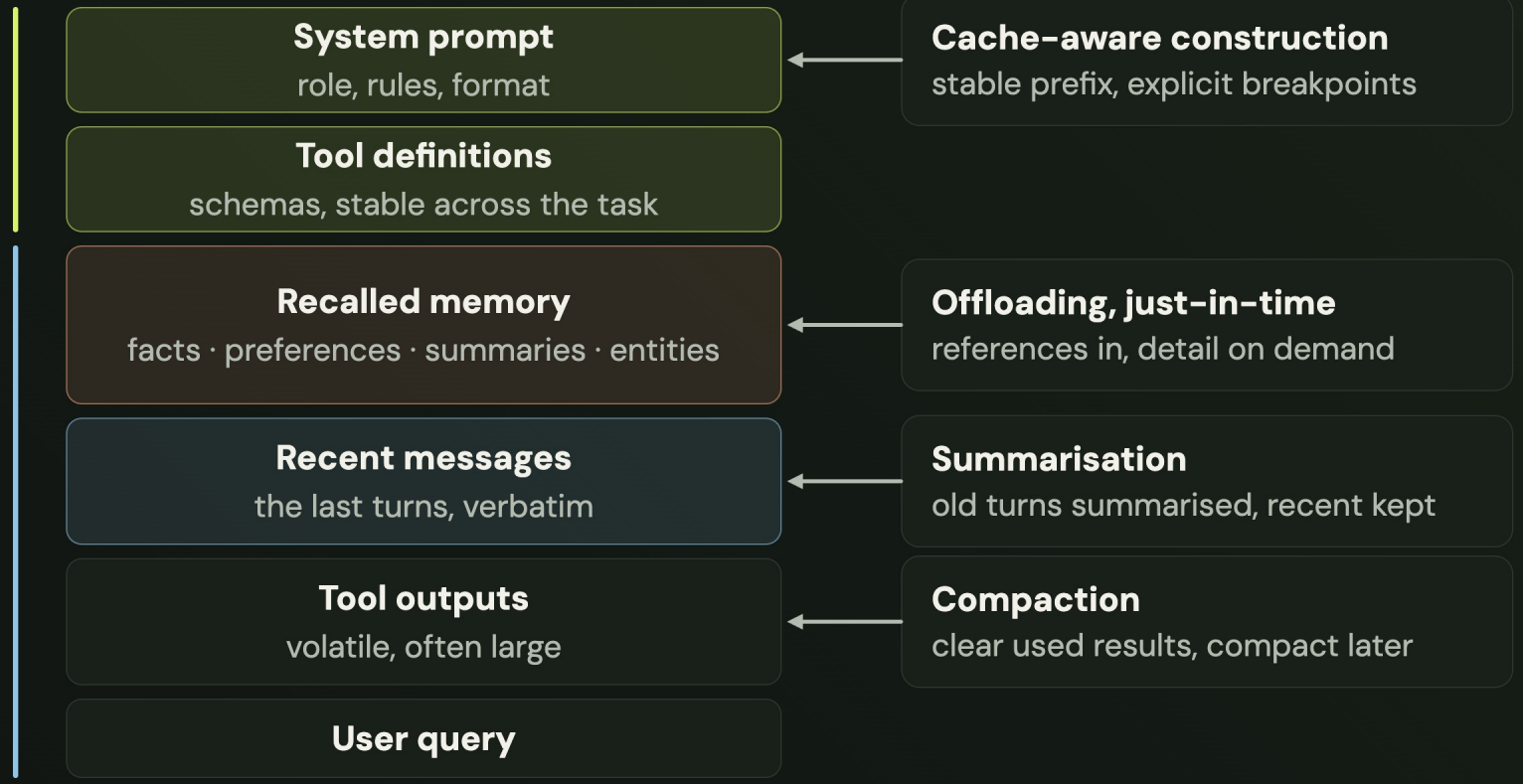
Context engineering curates and maintains the optimal set of information in the context window during inference.

STABLE PREFIX

Identical on every call. The prompt cache serves it, so it is nearly free after the first request.

ASSEMBLED PER CALL

The harness and the memory manager decide what goes here, how much of it, and in what order.



03 · CONTEXT ENGINEERING · OFFLOADING

Offloading: window, database, operations

The window is a cache of the database, not the other way round. Offloading keeps the cache small and the record complete.

03 · CONTEXT ENGINEERING · OFFLOADING

When utilisation crosses the threshold

BEFORE · turn 23

86% used · over the 80% threshold

System prompt	2k
Tool definitions	6k
Recalled memory	18k
Messages	40k
Tool outputs	106k
User query	0.4k

Trigger: utilisation above 80%, or one segment above 10% of the budget.

Candidate: the largest segment with the lowest utility: already used, old, and restorable from a reference.

Action: summary and reference in, payload out.

`tool_log · tl_8821`

summary · 120 tokens

payload · 106k tokens, the full result

`/runs/23/tool-outputs.json`

AFTER · same turn

33% used · 106k freed, record intact

System prompt	2k
Tool definitions	6k
Recalled memory	18k
Messages	40k
Reference stub · tl_8821	0.06k
User query	0.4k

The payload is one fetch away. The window keeps the 120-token summary and the id, which is all the next step usually needs.

03 · CONTEXT ENGINEERING · JUST IN TIME

Recalling what was offloaded, without refilling the window

01
Step 31 needs it
the plan references rows 40 to 60 of the result

→

02
fetch(tl_8821, rows=40..60)
a slice, not the whole payload

→

03
Headroom check
200k, less 16k reserve, less 94k used: 90k free; 4k asked

→

04
Inject for this step
into the tool-result slot, tagged ephemeral

→

05
Use it, then compact
the slice collapses back to the stub

HOW A RECALL IS STOPPED FROM BLOWING THE WINDOW UP AGAIN

Headroom before inject
Limit, minus output reserve, minus what is in use. A recall gets no more.

Compact form first
Summary and id first; the payload only when a step asks for a slice.

Budget and threshold
A token budget per recall, and a rerank threshold, not a fixed top-k.

Ephemeral injection
The slice lives for the step that fetched it, then compacts back.

Re-offload on use
A result that pushes past the threshold is offloaded again, same policy.

The offload policy and the recall policy read the same fields: tokens, utility, age, and a reference that restores.

03 · CONTEXT ENGINEERING

How compaction works in the Claude SDK

01
Threshold check
After each response: input, cache writes, cache reads and output tokens.

02
Summary generation
A summary prompt is injected; Claude writes a structured summary.

03
Context replacement
The SDK replaces the whole message history with that summary.

04
Continuation
The conversation resumes from the summary where it left off.

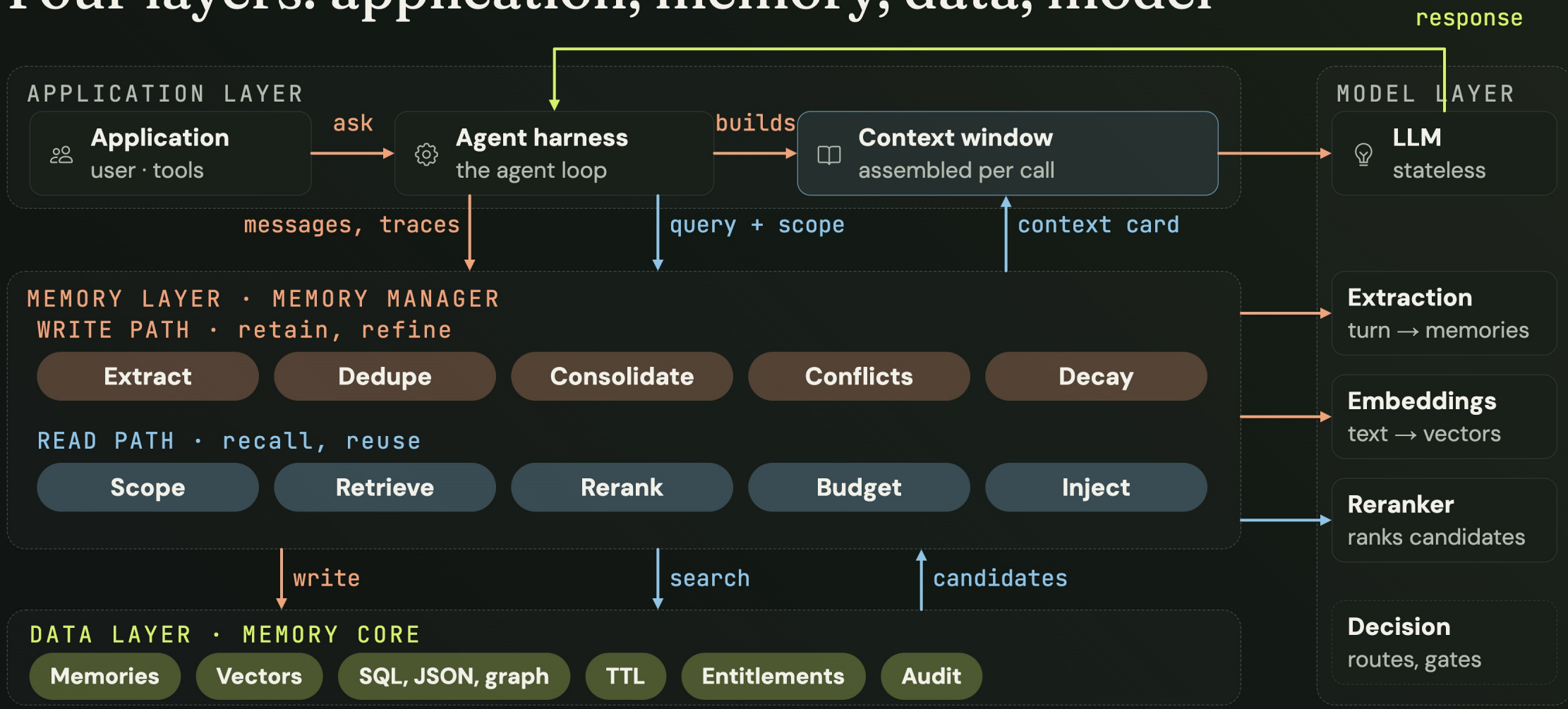
The next request pays a fresh cache write, and anything not in the summary or in memory is gone.

Enabled with compaction_control on tool_runner; the API also offers server-side compaction. Source: Claude Platform docs, compaction.

The context window is working memory. Treat it like a database and you pay for it on every turn.

O3 · REFERENCE ARCHITECTURE

Four layers: application, memory, data, model



03 · REFERENCE ARCHITECTURE

The read path decides cost and accuracy every turn

Step	What happens	Runs on	Serves	Cost note
Scope	Filter by user, tenant, entitlements and type before any similarity	Database filters	Recall	Cheapest step; do it first
Retrieve	Hybrid search: vector, keyword and graph over what scope left	Embeddings, database	Recall	Latency is the risk
Rerank	Score each candidate for relevance and applicability	Reranker or decision model	Recall, refine	Accuracy lives here
Budget	Keep only what fits this turn's token budget	Harness code	Reuse	Every item kept costs tokens every turn
Inject	Place the context card after the cache breakpoint	Harness code	Reuse	Wrong order breaks the prompt cache

03 · REFERENCE ARCHITECTURE

The write path decides what the agent becomes

Step	What happens	Runs on	Serves	Failure if skipped
Extract	Turn a trace into typed memories with a scope and a source	Extraction LLM	Retain	Memory is a transcript dump
Dedupe	Find similar units; drop what is already stored	Embeddings, decision model	Refine	Duplicates crowd recall
Consolidate	Merge and update; keep the old version with its validity window	LLM or code	Refine	Lost memories, no restore
Conflicts	Detect contradictions; decide which wins, or flag it	Decision model	Refine	Stale facts outrank fresh ones
Decay	Expire, demote or soft-delete on a TTL	Scheduled job	Refine	The store only grows

03 · MEMORY ENGINEERING · WORKED EXAMPLE

One exchange in, six typed memories out

THREAD t_2041 · TRAVEL AGENT · TURNS 1 TO 3

user

Book me for Web Summit in Lisbon, 9 to 12 November. Hotel near the venue, under €200 a night. I'm vegetarian and I like an aisle seat.

assistant

On it. Any timing preferences for the flights?

user

Nothing before 9am, the 6am flight last time was a disaster. Ana from my team joins on the 11th. Use the company card ending 4417.

DROPPED, NOT STORED

The card number and the pleasantries. Payment details never become memory.

extract →

preference — **Vegetarian; prefers an aisle seat**
Semantic · Entity memory · user: u_314 · turn 1

trip — **Lisbon trip; hotel near venue, under €200**
Semantic · Entity memory · trip: lis-nov · 9 to 12 Nov

fact — **Ana, a teammate, joins on 11 November**
Semantic · Entity memory · entity: ana · trip: lis-nov

rule — **Never book flights before 9am**
Procedural · Workflow memory · user: u_314 · turn 3

episode — **The 6am flight on the last trip went badly**
Episodic · Summary memory · ttl: 180 days · turn 3

rule — **Bill work travel to the company card**
Procedural · Workflow memory · org policy · no card number

03 · THE MODELS BEHIND MEMORY

One agent, eight kinds of model touching its memory

LLM

the agent's main model

USES read write

Reasons over recalled memory, and can save or edit memory through a memory tool.

Extractor

a small, cheap LLM

WRITES write

Turns a turn or a trace into typed memories with a scope, a type and a source.

Embedding model

vectors for search

ENCODES read write

Turns memories and queries into vectors, so similar things can be found and deduplicated.

Reranker

a cross-encoder

ORDERS read

Rescores retrieved candidates against the query, so only the best few spend the budget.

Decision model

System One, milliseconds

GATES read write

Fast calls at the gates: recall or not, store or skip, add, update or ignore.

Summariser

an LLM, or server compaction

COMPRESSES read write

Folds older turns and long tool results into a summary that fits the window.

Consolidator

a stronger LLM, offline

MERGES write

Between sessions: merges duplicates, resolves conflicts and decays stale memory.

Guard

a PII and safety classifier

FILTERS write read

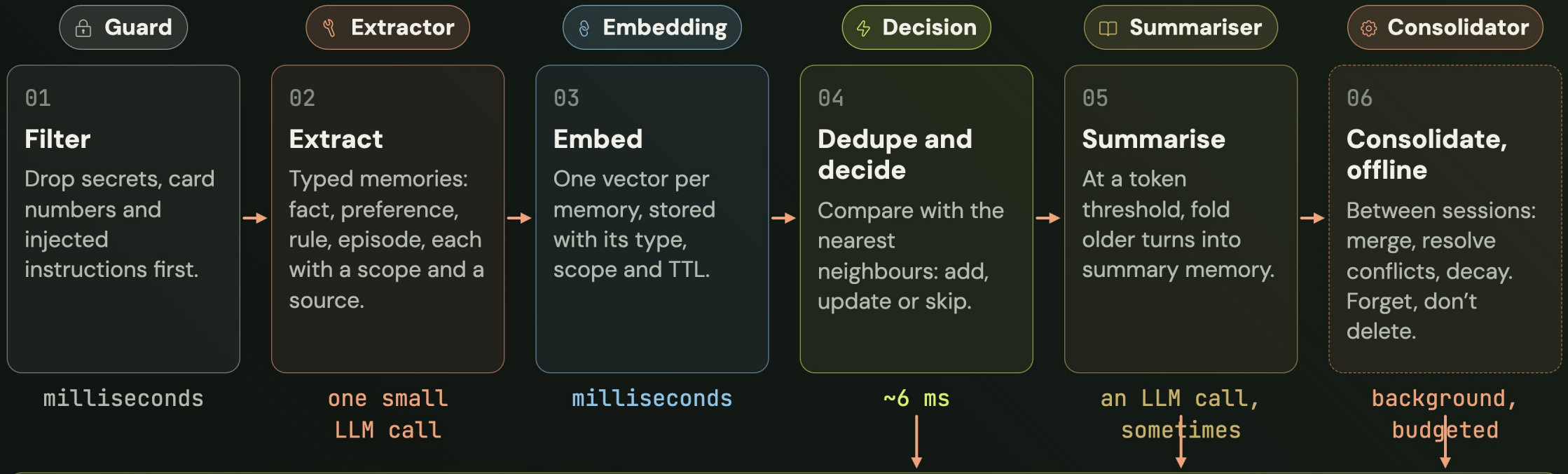
Stops what must never be stored, and instructions hidden in recalled memory.

Only the LLM-backed ones are expensive: the main call, extraction, summarisation and consolidation. The rest run in milliseconds.

03 · THE MODELS BEHIND MEMORY · WRITE PATH

The write path, model by model

IN a finished turn: messages, tool calls and results

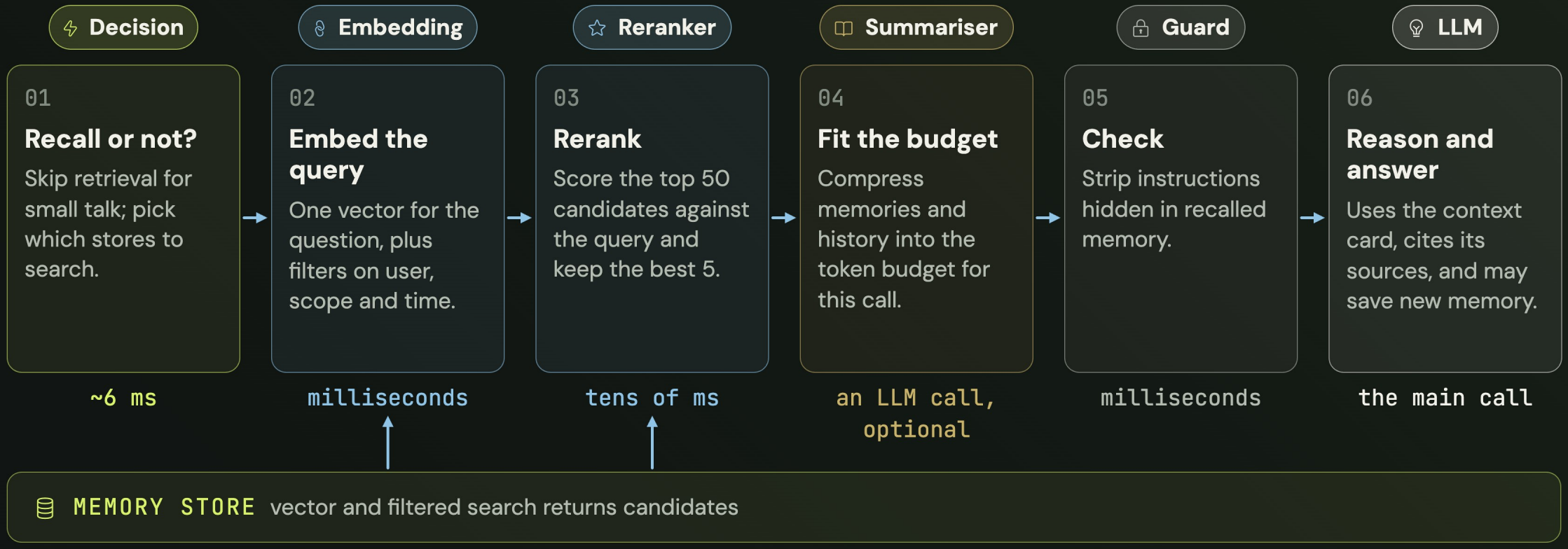


OUT typed, embedded, deduplicated memory · run it out of band, off the user's latency

03 · THE MODELS BEHIND MEMORY · READ PATH

The read path, model by model

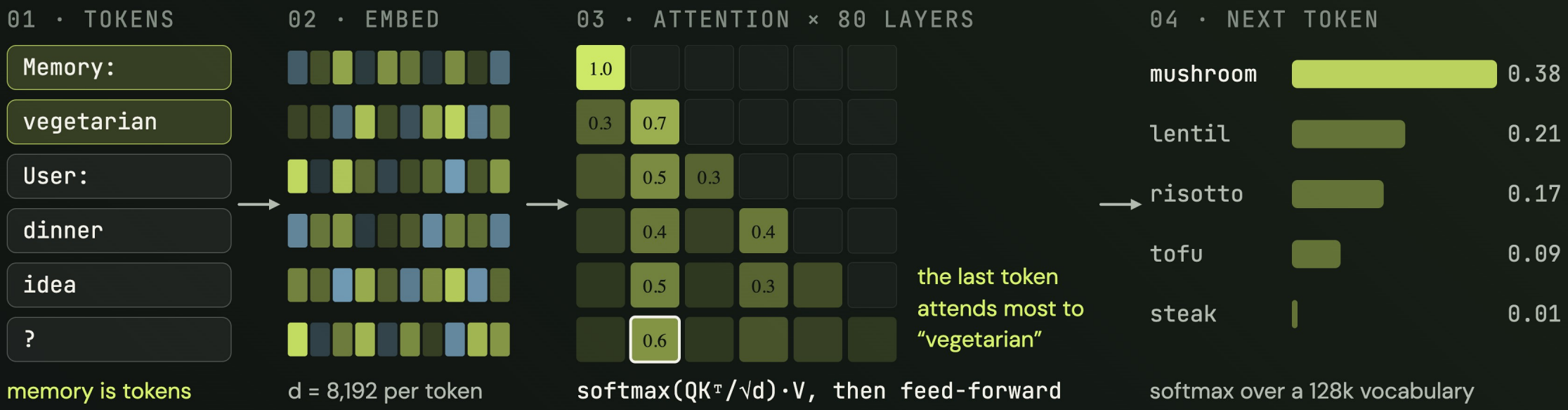
IN the user's next message



OUT an answer grounded in memory · everything before step 06 should cost less than the answer

03 · INSIDE THE MODELS

Inside the LLM: how memory in the window gets used



140 GFLOPs

per generated token for a 70B model: about 2 × the parameters.

n^2

attention compares every token with every earlier one, so long windows cost more than linearly.

KV cache

K and V for the prefix are kept between calls: the prompt cache you rent is exactly this.

Illustrative sizes from Llama 3 70B: d = 8,192, 80 layers, 64 heads, 128k vocabulary.

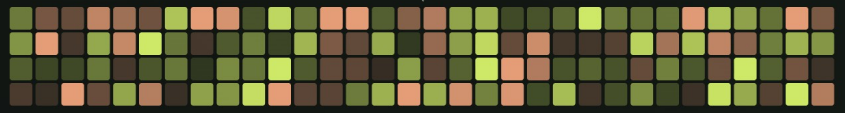
03 · INSIDE THE MODELS

Inside the embedding model: memory as points in space

01 · TEXT

“Prefers an aisle seat”

Embedding model · 1,024 dimensions



[0.12, -0.07, 0.31, …] · 128 of 1,024

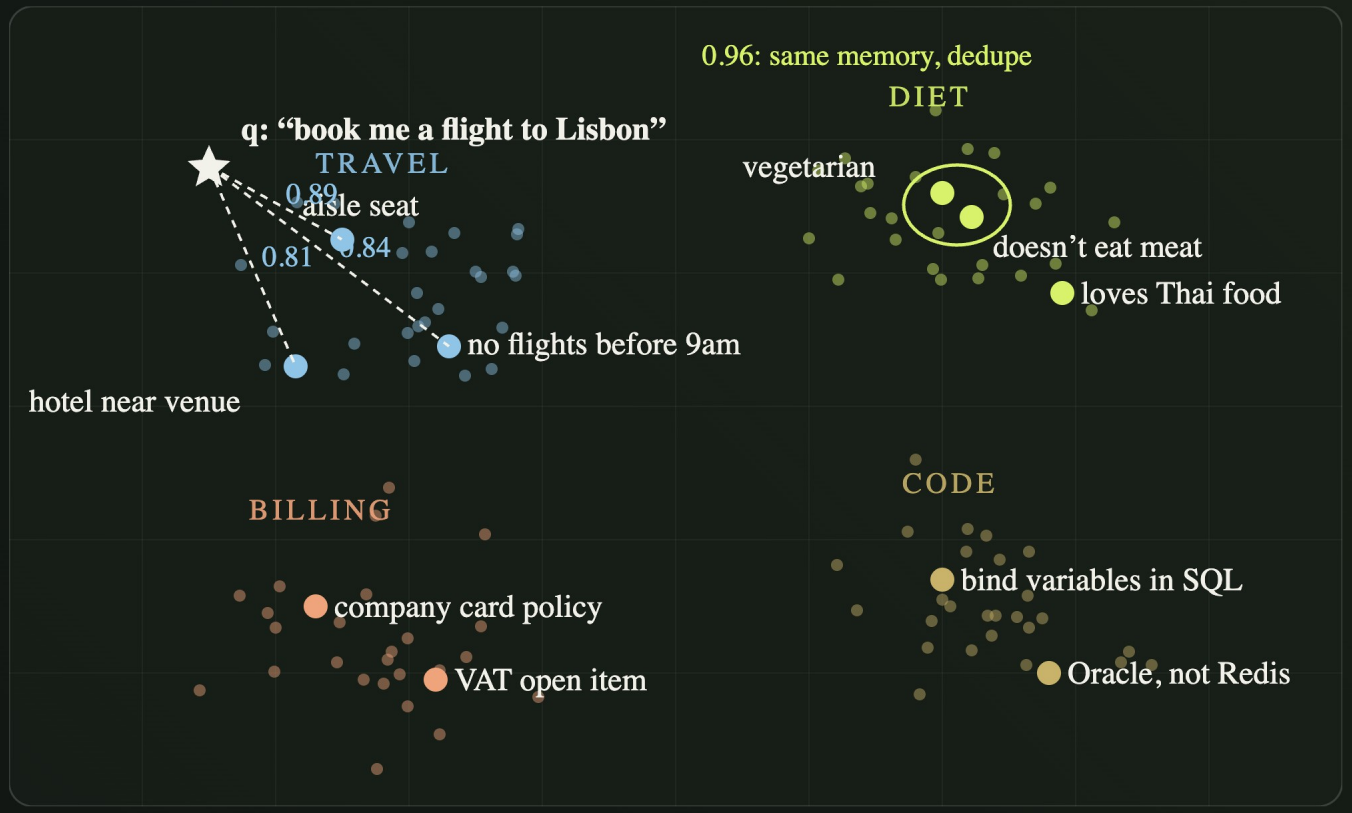
COSINE SIMILARITY

cos(q, m) = q·m / (‖q‖ ‖m‖)

q=[0.6, 0.8, 0.0] m=[0.5, 0.7, 0.5]

q·m = 0.30 + 0.56 + 0 = 0.86

‖q‖ = 1.0 ‖m‖ ≈ 0.99 → cos ≈ 0.86

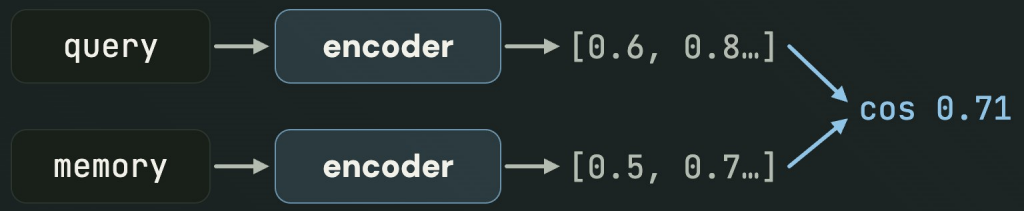


A 2D projection of 1,024 dimensions. Near means similar in meaning: recall takes the nearest, dedupe merges the overlapping.

03 · INSIDE THE MODELS

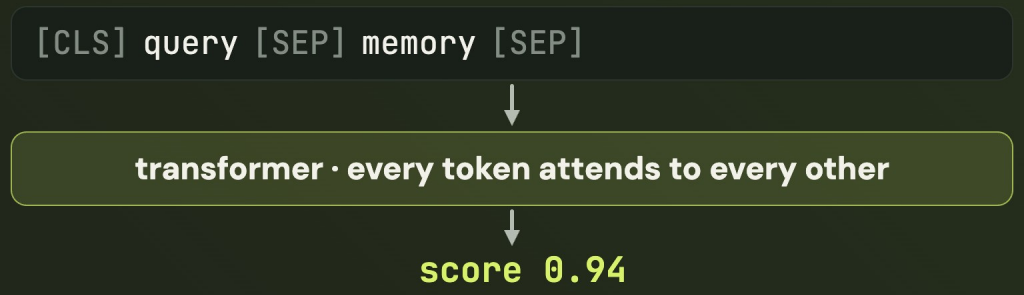
Inside the reranker: reading query and memory together

BI-ENCODER · the embedding search



Each side is encoded alone, ahead of time. Fast over millions of memories, but it never sees how the words relate.

CROSS-ENCODER · the reranker



Reads both together. Accurate, but one forward pass per pair.

q: “Can I claim the bereavement fare after I fly?”

vector rank	candidate memory	rerank score	new rank
#1	Bereavement leave policy for employees	0.08	dropped
#2	Refund requests after travel: how to apply	0.62	2
#3	Bereavement fares must be requested before travel	0.94	1
#4	Fare classes and change fees	0.21	3
#5	Travelling with a medical condition	0.05	dropped

03 · INSIDE THE MODELS

Inside a decision model: fast calls at the memory gates

gate	input	decision · confidence
Recall?	"hi, morning!"	NO RECALL
Store?	"Thanks, that's all for today."	SKIP
Store?	"Never book flights before 9am."	STORE · rule
Conflict?	new: Acme on enterprise old: Acme on starter	SUPERSEDE
Type?	"The 6am flight last time was a disaster."	ESCALATE · 0.64

Below the 0.7 threshold, the decision model hands off to the LLM. Calibrate the threshold on your own labelled traces.

SYSTEM ONE VS SYSTEM TWO

	Decision model	LLM
Size	1.92B params	frontier
Latency	~6 ms	seconds
Input price	$0.02 / M	$4 / M
Output	label + score	open text
Job	gates, at volume	reasoning

200× cheaper

per input token, and fast enough to sit on every turn without the user noticing.

Liquid d1-2B: 5.7 ms on an H100; hosted d1.1 at $0.02 per million tokens. Opus 5.5 input: $4 per million.

03 · MEMORY ENGINEERING

Three principles of memory engineering

PRINCIPLE 01

The RBCs

Reliability, believability and capability: what memory should improve in an agent, and how you judge it.

PRINCIPLE 02

Forget, don't delete

Retire memory with a validity window and provenance, so you can audit, restore and trace what was learned from it.

PRINCIPLE 03

The 4 Rs

Retain, reuse, refine, recall: the operations every memory system performs, and the vocabulary for the rest of today.

03 · MEMORY ENGINEERING

The 4 Rs of memory engineering

R1

Retain

Decide what is worth writing, and write it before the window clears.

R2

Reuse

Serve the same knowledge again instead of paying to re-derive it.

R3

Refine

Consolidate, correct and promote as evidence arrives. Forget, don’t delete.

R4

Recall

Bring back the right memory at the right moment, ranked by relevance and recency.

03 · MEMORY ENGINEERING

Each R is an operation, a cost lever and a failure

R	Memory operation	Token lever	What breaks without it
Retain	Extract and write	Promotion gates	The agent forgets it already tried that
Reuse	Procedures and caching	Prompt caching	Paying to re-derive the same answer
Refine	Consolidate and forget	A smaller memory block	Stale and contradictory memories
Recall	Retrieve and rerank	Retrieval budgets	An old turn treated as the current one

MODULE 04

Tokenomics

Your agent’s bill is mostly tokens it re-reads. Memory decides which ones.

04 · TOKENOMICS

What one model request costs

COST = cached tokens × read price + newly cached × write price + fresh input × input price + output × output price

TOKENS IN THE REQUEST · 23,500

WHAT YOU PAY · $0.028

Cached prefix

20,000 tokens read from the cache

× $0.20 per million

= $0.004

Cache write

2,000 tokens written for next time

× $5 per million

= $0.010

Fresh input

1,000 new tokens after the breakpoint

× $4 per million

= $0.004

Output

500 tokens generated

× $20 per million

= $0.010

Same request with no cache: 23,000 input tokens × $4 per million, plus output, is $0.102. Opus 5.5 list prices, Nov 2026.

04 · TOKENOMICS

One request on Opus 5.5 costs 83% less with caching

Term	Tokens	Price per million	Cost
Cache reads (stable prefix)	50,000	$0.20	$0.0100
Cache writes (last turn)	2,000	$5.00	$0.0100
Uncached (new message)	300	$4.00	$0.0012
Output	800	$20.00	$0.0160
Total with caching	53,100		$0.0372
Same request, no caching	53,100		$0.2252

The cached prefix also cuts time to first token. Output is now the biggest line, so the next lever is shorter answers and fewer turns.

Illustrative token counts. Uncached: 52,300 input tokens at $4 per million plus 800 output tokens at $20 per million.

04 · TOKENOMICS

A 50,000-token prefix over a 40-turn session

$8.00

No caching: 40 × 50,000 tokens at $4 per million

$0.64

1 write at $5, 39 reads at $0.20 per million

A 92% cut on the prefix, before any memory engineering.

Claude Opus 5.5 list prices, November 2026.

04 · TOKENOMICS

The KV cache, and why long context is expensive

Prefill

The whole prompt is processed in parallel; every token's keys and values are computed, layer by layer.

KV cache

Keys and values are stored so they are computed once, not on every generated token.

Decode

One token at a time, each step reading the whole cache.

```
KV bytes per token = 2 × layers × KV heads × head dim × bytes per value
```

A 70B-class model with grouped-query attention holds about 320KB per token: roughly 33GB at 100,000 tokens.

Illustrative: 80 layers, 8 KV heads, head dimension 128, 16-bit values. GPU memory, not maths, is the bottleneck.

04 · TOKENOMICS

Where memory sits decides your cache hit rate

- **Tools**
- **System prompt**
- **Procedural memory and profile**
- **Retrieved memories**
- **Conversation**
- **Latest message**

Stable prefix, cached: reads at a fraction of input price

Volatile: full price, after the breakpoint

Stable before, volatile after

Per-turn memory goes after the last breakpoint, or every write invalidates the cache.

Write memory out of band

Consolidate between sessions, as Codex and ChatGPT do in the background, not mid-session.

Compaction resets the cache

A summary replaces the prefix and the next request pays a cache write.

04 · TOKENOMICS

The toolkit, and when to pull each lever

Lever	What it cuts	Pull it when
Prompt caching	Re-reading a stable prefix	First, on any agent with a long system prompt or tool list
Semantic caching	Whole calls for near-duplicate queries	High-volume assistants with repeated questions
Compaction and tool-result clearing	Growing conversation history	Long sessions near the window limit
Memory consolidation	Duplicate and stale memories	Long-term memory passes a few hundred items
Retrieval budgets and reranking	Oversized memory blocks	The memory block grows faster than quality
Model routing	Price per token	Repetitive, well-specified tasks with good evals

Order matters. Caching first because it is nearly free. Routing last because it needs evals.

MODULE 05

Memory for coding agents

Give Claude Code and Codex one memory they share, with the MemoRizz plugin.

05 · MEMORY FOR CODING AGENTS

Anatomy of a memory plugin: MemoRizz

CAPTURE

Hooks

Load memories at session start, save each turn, summarise each session.

TEACH

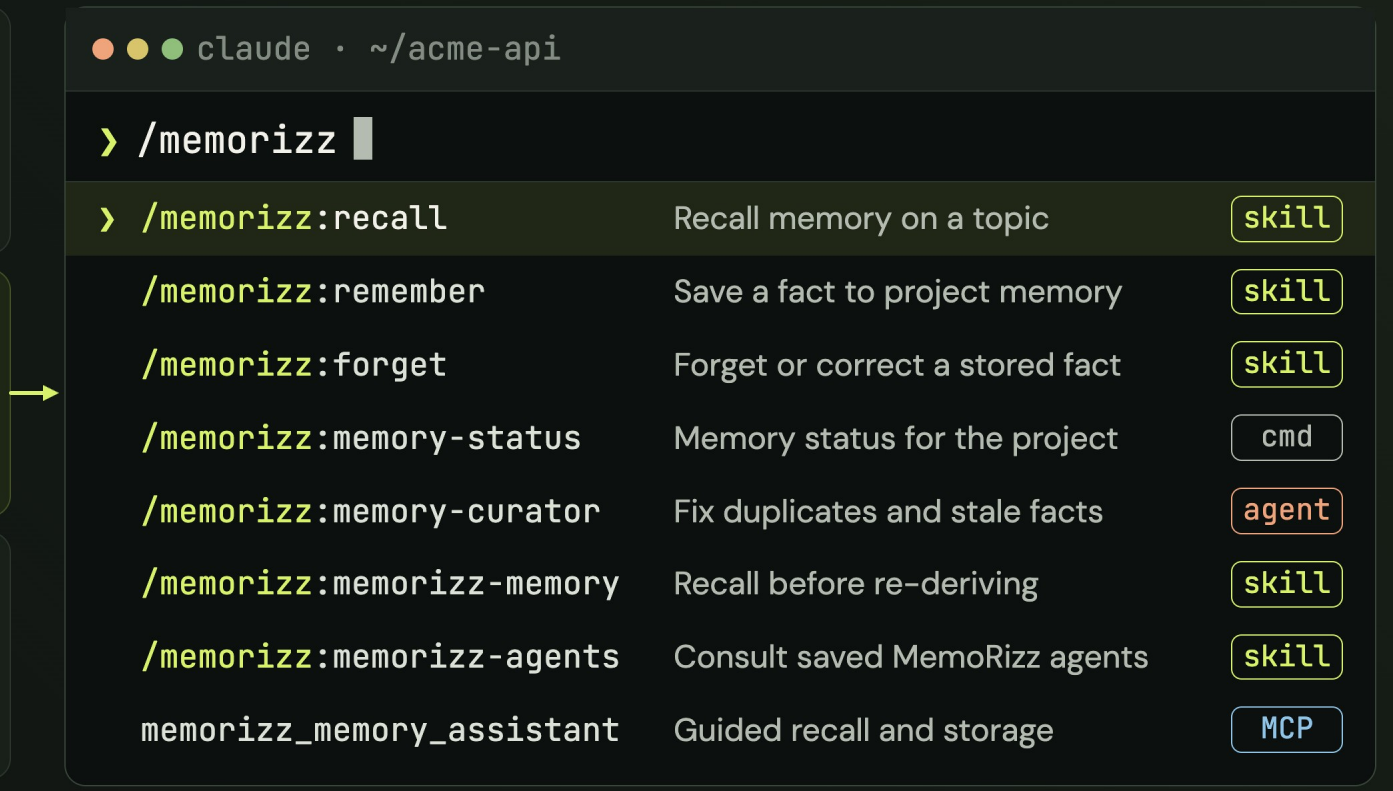
Skills

Recall, save, correct and curate. Slash commands and a curator subagent.

SHARE

MCP server

One memory per project, shared by Codex and Claude Code, local or hosted.



What Codex learns, Claude Code can recall, and the other way round.

05 · MEMORY FOR CODING AGENTS

Live: adding the MemoRizz plugin

CODEX

```
memorizz plugin install codex

codex plugin marketplace add RichmondAlake/memorizz
codex plugin add memorizz@memorizz
```

CLAUDE CODE

```
memorizz plugin install claude-code

claude plugin marketplace add RichmondAlake/memorizz
claude plugin install memorizz@memorizz
```

1. Show native memory in both tools
2. Install MemoRizz in Codex
3. Install it in Claude Code
4. Save in one, recall in the other
5. Measure tokens per turn, plugin off and on

Gotcha: Codex's memories.disable_on_external_context keeps chats that used MCP out of native memory generation.

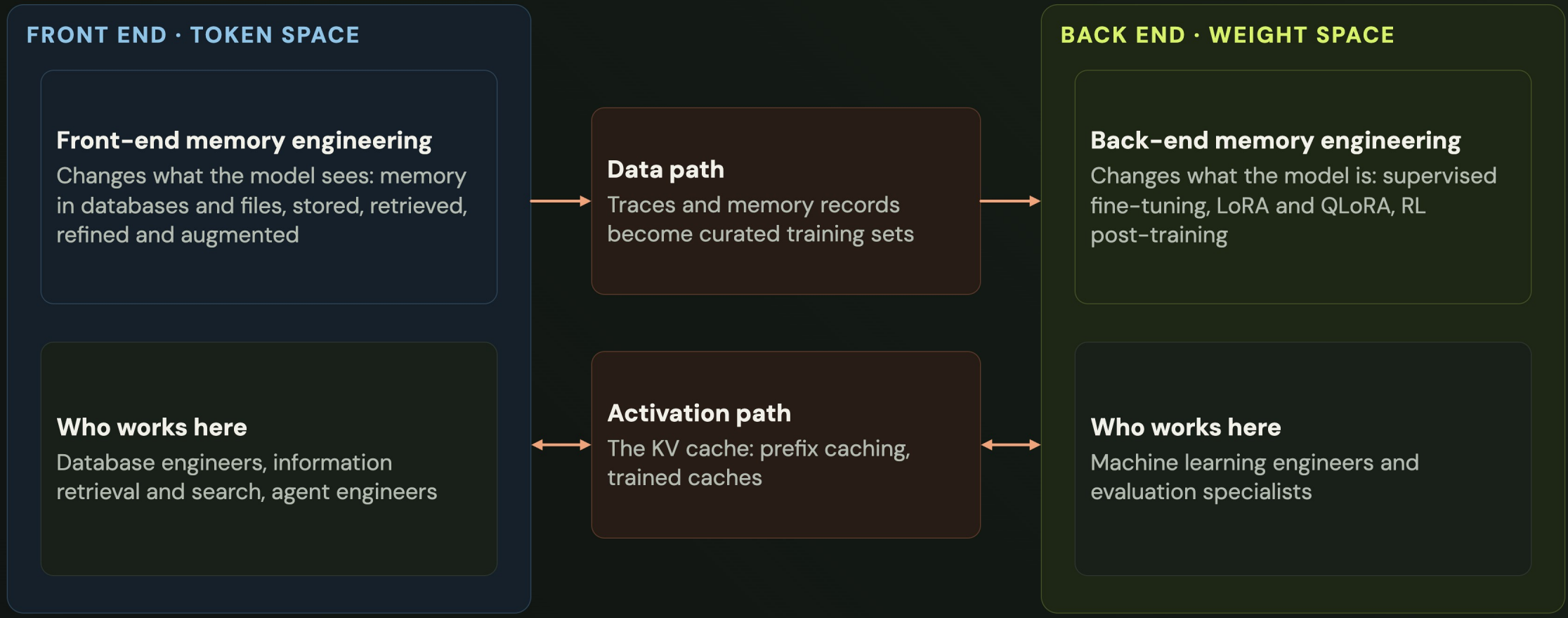
MODULE 06

Continual learning

Agent memory is the foundation of continual learning.

06 · CONTINUAL LEARNING

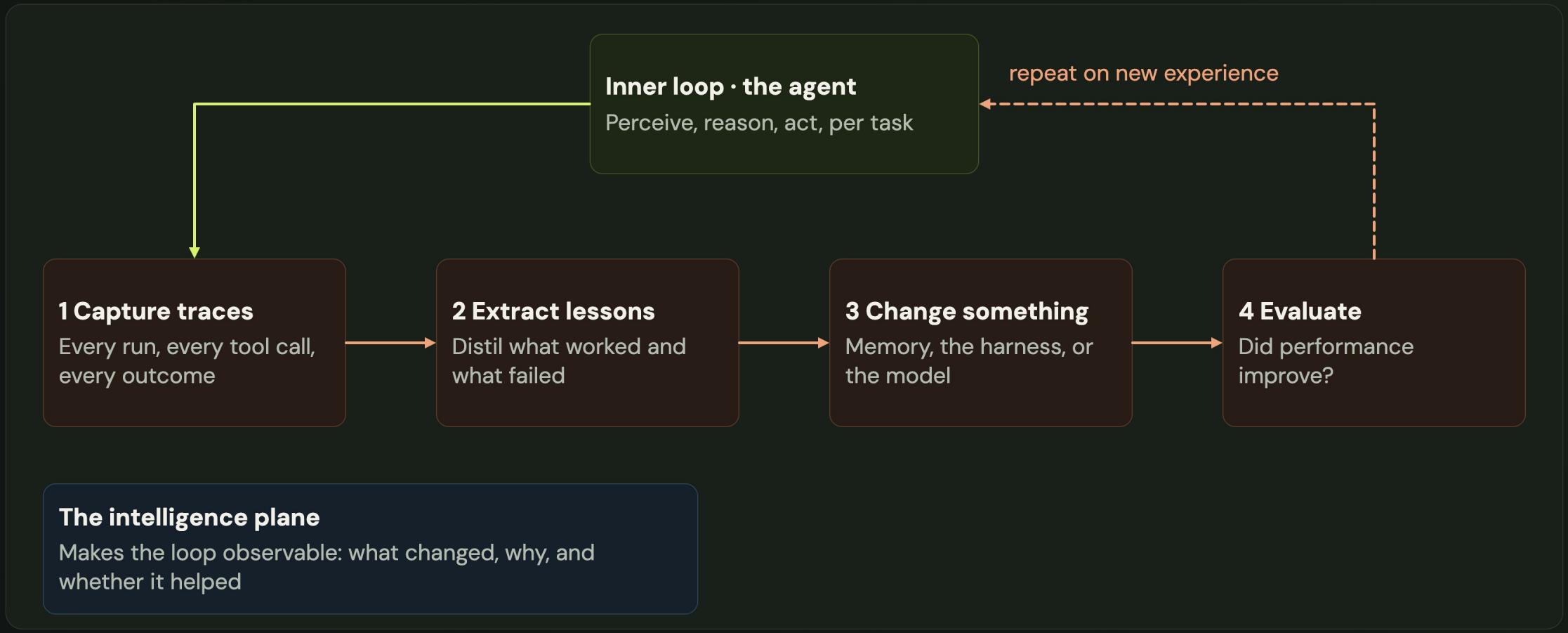
Memory engineering has two ends and two bridges



A single fine-tuning run is not continual learning. Continual learning repeats on new experience and must keep what it learned last time.

06 · CONTINUAL LEARNING

An inner loop that acts, an outer loop that learns



06 · CONTINUAL LEARNING

A rule for where a piece of knowledge belongs

KEEP IT IN THE FRONT END

If it...

- Changes often
- Belongs to one user or tenant
- Needs a source citation
- Must be deletable on request

GRADUATE IT TO THE WEIGHTS

If it...

- Is stable and broadly shared
- Costs context budget on every call
- Has a clean, permitted dataset
- Has an evaluation you trust

When behaviour graduates into the weights, the front end stops carrying it, and that context budget goes back to evidence the model cannot know.

06 · CONTINUAL LEARNING

Continual learning without touching the weights

01
Workflow memory
Every run stored with steps, outcome and lesson.

▸

02
Score
Frequency × success × recency, over canonical trajectories.

▸

03
Distil and validate
An LLM drafts a skill; a gate checks tools exist and rules hold.

▸

04
Skillbox
Promoted skills load before planning, above raw workflows.

▸

05
Monitor and demote
Drift or failures send a skill back down.

Workflow memory that proves itself becomes a skill. That is learning in token space, and it produces the dataset the back end needs later.

This is the promotion design in MemoRizz: workflow memory feeds a skillbox, with a validation gate and a demotion path.

KEEP GOING

After today

NEXT COHORT

Tuesday 15 December 2026

Introduction to AI Agent Memory, Cohort 3, on Maven.

FREE

100 Days of Agent Memory

Days 1 to 10 of the ebook, on Maven and llmcamp.com.

OPEN SOURCE

MemoRizz

github.com/RichmondAlake/memorizz: the plugin and library from today.

You leave with the recording, the reference architecture, the notebook and the bonus material on MemoRizz and papers worth reading.